Cross-fitted marginal-estimand PC/TMLE analysis

Complete methodology, inference, multiplicity adjustment, and saved study figures

---

Purpose. This report documents the marginal-estimand branch for the fixed 12-gene panel and the all-999-gene study. The target is a symmetric log interaction of the marginal joint distribution of a selected gene set. It is an association among binary expression states, not a causal effect, direct regulatory claim, or biological mechanism. The final appendix reproduces both analysis guides verbatim, preserving every guide detail: commands, study configuration, runtime, files, support rules, scalability caveats, and figure interpretation.

Contents

1 Data, PC fitting, and cross-fitting **2**

2 Marginal estimand and exact probability/count calculation **2**

3 TMLE update, score, and EIF **3**

4 Unregularized support, uncertainty, p-values, and q-values **3**

5 Results: 12-gene study **4**

6 Results: 999-gene study **19**

A Complete 12-gene analysis guide (verbatim) **23**

B Complete 999-gene analysis guide (verbatim) **29**

1 Data, PC fitting, and cross-fitting

The source CSV contains 11,950 cells and 1,000 numeric columns; its constant unnamed first metadata column is removed, leaving 999 binary named gene columns. The 12-gene panel has a fixed analysis order:

`Spc24, Gmnn, Prim2, Gins2, Cks2, Cdca8, Pbk, Ube2c, Ccna2, Apitd1, Hells, Mcm6`.

No cells or genes are filtered. A SHA-256 fingerprint of ordered gene names and binary values is stored with every checkpoint and result. Reduced-sample runs deliberately change this fingerprint so they cannot reuse full-data artifacts.

Architecture selection is separate from inference. With seed 0, 9,560 cells are used for selection training and 2,390 for validation; candidates are ranked by mean validation negative log likelihood. The 12-gene selection evaluates 39 combinations of four PC architectures, component counts, and learning rates, selecting a 32-component mixture-Chow–Liu model (validation NLL 4.081627). A mixture tree PC has the form

$$p(x) = \sum_z \pi_z p(x_{\text{root}} \mid z) \prod_{j \neq \text{root}} p\{x_j \mid x_{\text{parent}(j)}, z\}.$$

For tree candidates, the Chow–Liu structure is learned from selection-training cells only using smoothed mutual information and a maximum-spanning tree. No interaction effect, p-value, or q-value is used in selection.

The selected specification is then fitted separately across a two-fold split of 5,975 held-out cells per fold. PC_1 learns from fold 2 and evaluates fold 1; PC_2 learns from fold 1 and evaluates fold 2. Thus every result uses a PC that did not train on the evaluated cell. Fold estimates are combined using held-out-fold sample-size weights and EIF moments are pooled. For 12 genes, final PCs use the 32-component mixture-Chow–Liu architecture, 20 epochs, 128-cell minibatches, learning rate 0.02, weight decay 10^{-5}, and CPU. The 999-gene final configuration is retained in its guide and uses a 32-component latent-class residual tree.

2 Marginal estimand and exact probability/count calculation

For selected gene set S of order r, let $a \in \{0,1\}^r$ denote a binary expression pattern and define $s(a) = (-1)^{r-\sum_j a_j}$. The marginal interaction estimand is

$$\theta_r(p) = \sum_{a \in \{0,1\}^r} s(a) \log p(X_S = a).$$

For a pair this reduces to the log odds ratio

$$\theta_2(p) = \log \frac{p_{00} p_{11}}{p_{01} p_{10}},$$

where $p_{ab} = p(X_i = a, X_j = b)$. Zero means no interaction on this scale; a positive or negative value is positive or negative marginal association. For larger sets, exponentiating produces an alternating cross-product ratio.

The PC obtains every selected-set pattern probability exactly by tree message passing: it fixes selected states, sums over all other tree nodes, and averages over mixture components. It does not simulate cells. Held-out pattern counts are created in bulk by binary encoding plus `torch.bincount`. In the all-999-gene pairwise study, a special exact cache holds $p_i = p(X_i = 1)$, $p_{11,ij} = p(X_i = 1, X_j = 1)$, observed $n_i, n_{11,ij}$, and fold sizes for each of 498,501 unordered pairs. The other cells follow from

$$\begin{aligned} p_{00} &= 1 - p_i - p_j + p_{11}, & n_{00} &= n - n_i - n_j + n_{11}, \\ p_{01} &= p_j - p_{11}, & n_{01} &= n_j - n_{11}, \\ p_{10} &= p_i - p_{11}, & n_{10} &= n_i - n_{11}. \end{aligned}$$

This analytic cache makes exhaustive pairs practical without enumerating a 2^{999} state space; it does not generalize to exhaustive higher orders.

3 TMLE update, score, and EIF

Within each held-out fold and target, the initial PC probability table $p_0(a)$ is updated in the signed fluctuation submodel

$$p_\epsilon(a) = p_0(a) + \epsilon s(a).$$

The feasible ϵ interval maintains strict positivity of all cells. For a pair with $s = (+1, -1, -1, +1)$,

$$\max(-p_{00}, -p_{11}) < \epsilon < \min(p_{01}, p_{10}).$$

After moving endpoints slightly inward, 64 vectorized bisection iterations solve the empirical EIF score equation

$$\sum_a p_n(a) \frac{s(a)}{p_\epsilon(a)} = 0.$$

TMLE therefore chooses a probability-table update with empirical EIF mean zero; it does *not* minimize the EIF. At the targeted table p^*, the centered EIF is

$$H(a) = \frac{s(a)}{p^*(a)} - \mathbb{E}_{p^*}\left\{ \frac{s(A)}{p^*(A)} \right\}.$$

The updated fold log interactions are combined with held-out sample-size weights. This targeting, exact PC probability calculation, and bulk count construction are vectorized across targets and folds.

4 Unregularized support, uncertainty, p-values, and q-values

Two result states are intentionally distinct. A target is *targeted* when its initial table is finite and strictly positive and the bisection equation has a finite root in both folds. It is *supported for unregularized inference* only when it is targeted and, in each fold, all 2^r observed patterns meet `minimum-cell-count`, all initial PC probabilities meet `minimum-probability`, and the table is finite. Targeted-but-unsupported estimates remain descriptive but get no SE, CI, p-value, or q-value. The support rules are gates: no pseudocount, continuity correction, probability floor, ridge penalty, or variance regularization alters counts/probabilities.

Let N be the pooled held-out sample size. The pooled EIF standard error is

$$\widehat{\mathrm{SE}}(\theta) = \sqrt{\frac{\sum_i H_i^2 - (\sum_i H_i)^2/N}{(N-1)N}}.$$

The table reports $\theta \pm 1.959964\,\widehat{\mathrm{SE}}$, and the two-sided normal p-value for $H_0 : \theta = 0$ is

$$p_h = 2\Phi\Big(- \Big| \theta_h / \widehat{\mathrm{SE}}_h \Big| \Big).$$

For the supported family of m hypotheses, sort $p_{(1)} \le \cdots \le p_{(m)}$. This branch reports Benjamini–Hochberg (BH) adjusted p-values:

$$q_{(i)}^{\mathrm{BH}} = \min_{j \ge i} \min\left\{ 1, \frac{m}{j} p_{(j)} \right\}; \qquad p_{(k)} \le \frac{k\alpha}{m}.$$

The first expression is returned to each original target; the second is the BH selection condition at FDR α. Thus $q_h \le \alpha$ says the target is selected when BH is run at α, not that q_h is a posterior probability or a probability that an edge is false. BH's usual FDR guarantee requires valid null p-values plus independence or positive regression dependence.

For comparison, Benjamini–Yekutieli (BY), used on the conditional-estimand branch and suitable for arbitrary dependence, inserts the harmonic factor $c(m) = \sum_{\ell=1}^{m} 1/\ell$:

$$q_{(i)}^{\mathrm{BY}} = \min_{j \ge i} \min\left\{ 1, \frac{m\, c(m)}{j} p_{(j)} \right\}; \qquad p_{(k)} \le \frac{k\alpha}{m c(m)}.$$

The overview has $y_h = -\log_{10}(q_h)$, capped at 50 for display; $q = 0.05$ maps to 1.301. Display also requires $|\theta| \ge 0.25$, an effect-size rather than a multiplicity condition. The guide explains the forest, network, hypergraph, fallback, edge-cap, and higher-order-screening rules in full.

5 Results: 12-gene study

The saved full-panel run used `minimum-cell-count=1`, `minimum-probability=1e-12`, target batches of 512, and 64 bisections. It trained once (2.13 s complete command) and then ran each full order; total fresh sequential time was 23.26 s. The following complete-command timings include loading, queries, counts, targeting, inference, result saving, and figures.

Order	Targets	Interaction command	Fresh fit + one order
2	66	1.60 s	3.73 s
3	220	2.01 s	4.14 s
4	495	2.95 s	5.08 s
5	792	4.47 s	6.60 s
6	924	5.28 s	7.41 s
7	792	4.82 s	6.95 s

The saved plots below provide the available overview, forest, and network/hypergraph results for every completed panel order. Full tall labelled comparisons are retained in the study directories and described precisely in the verbatim guide appendix.

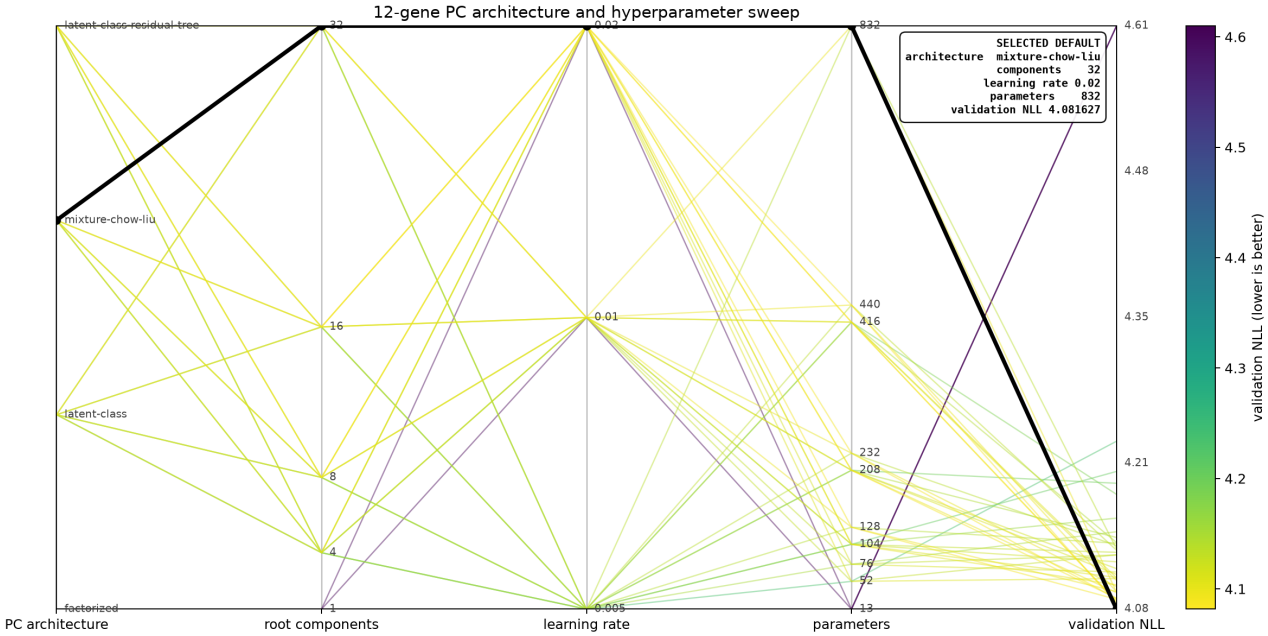


Figure 1: 12-gene PC architecture sweep retained for provenance.

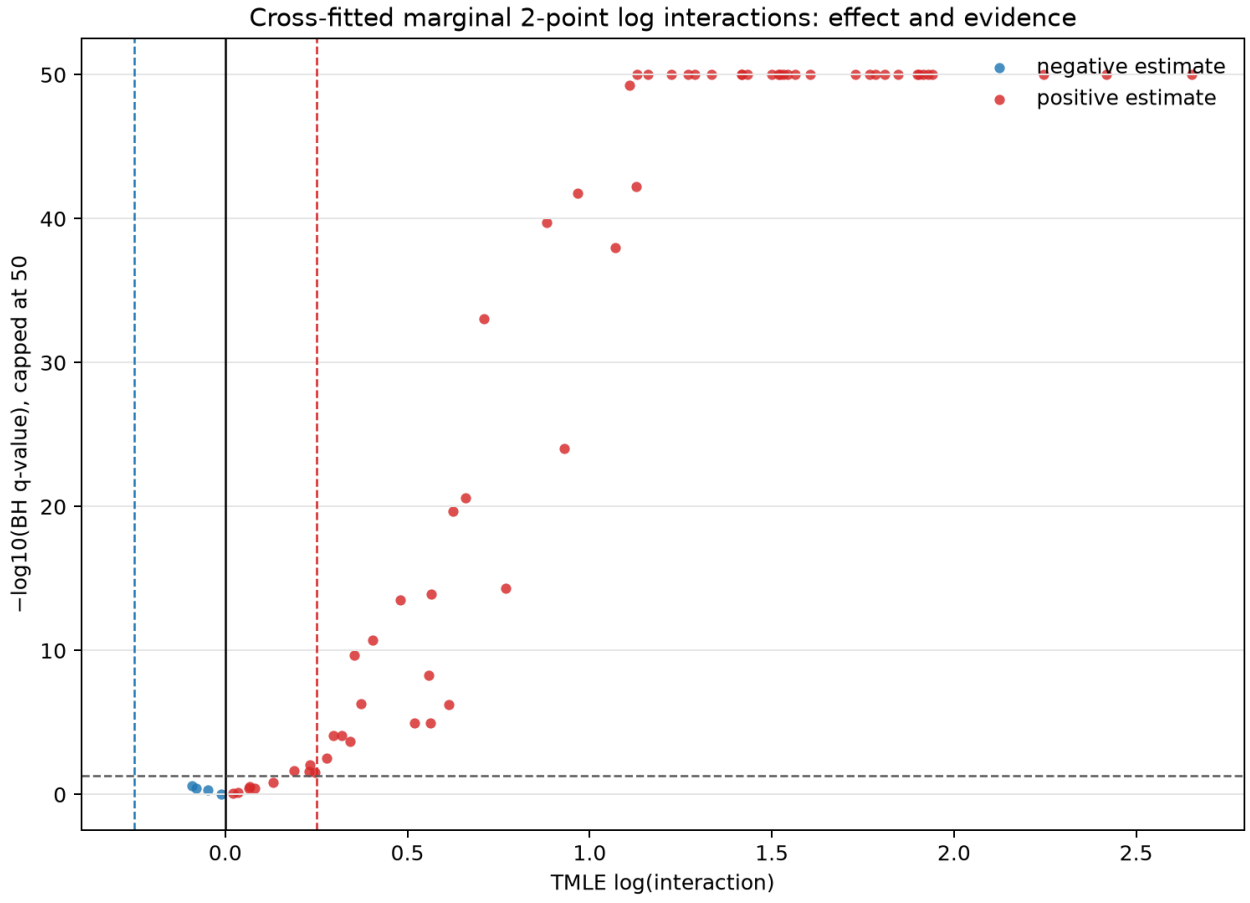


Figure 2: 12-gene marginal interactions, order 2: overview.

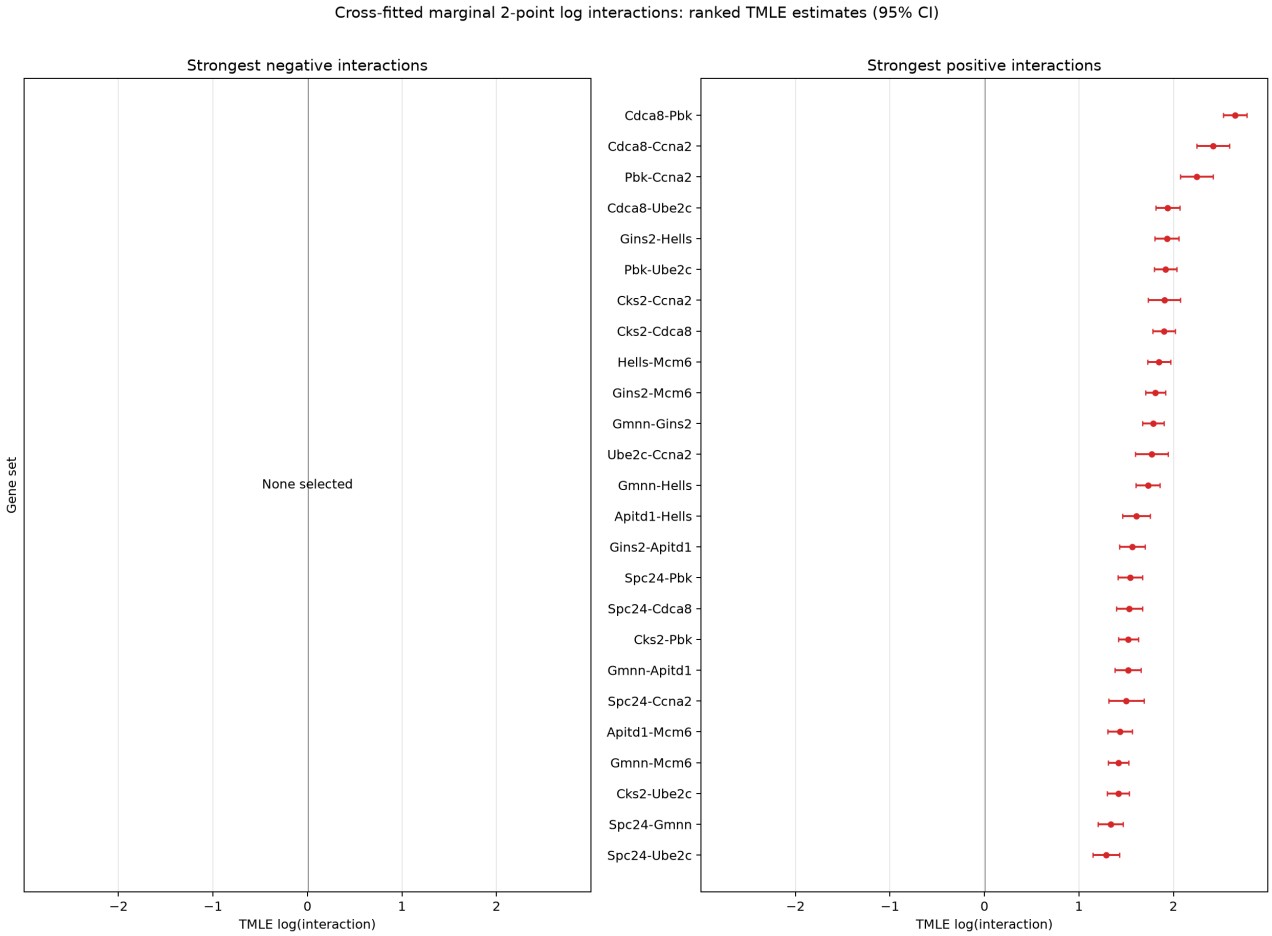


Figure 3: 12-gene marginal interactions, order 2: forest summary.

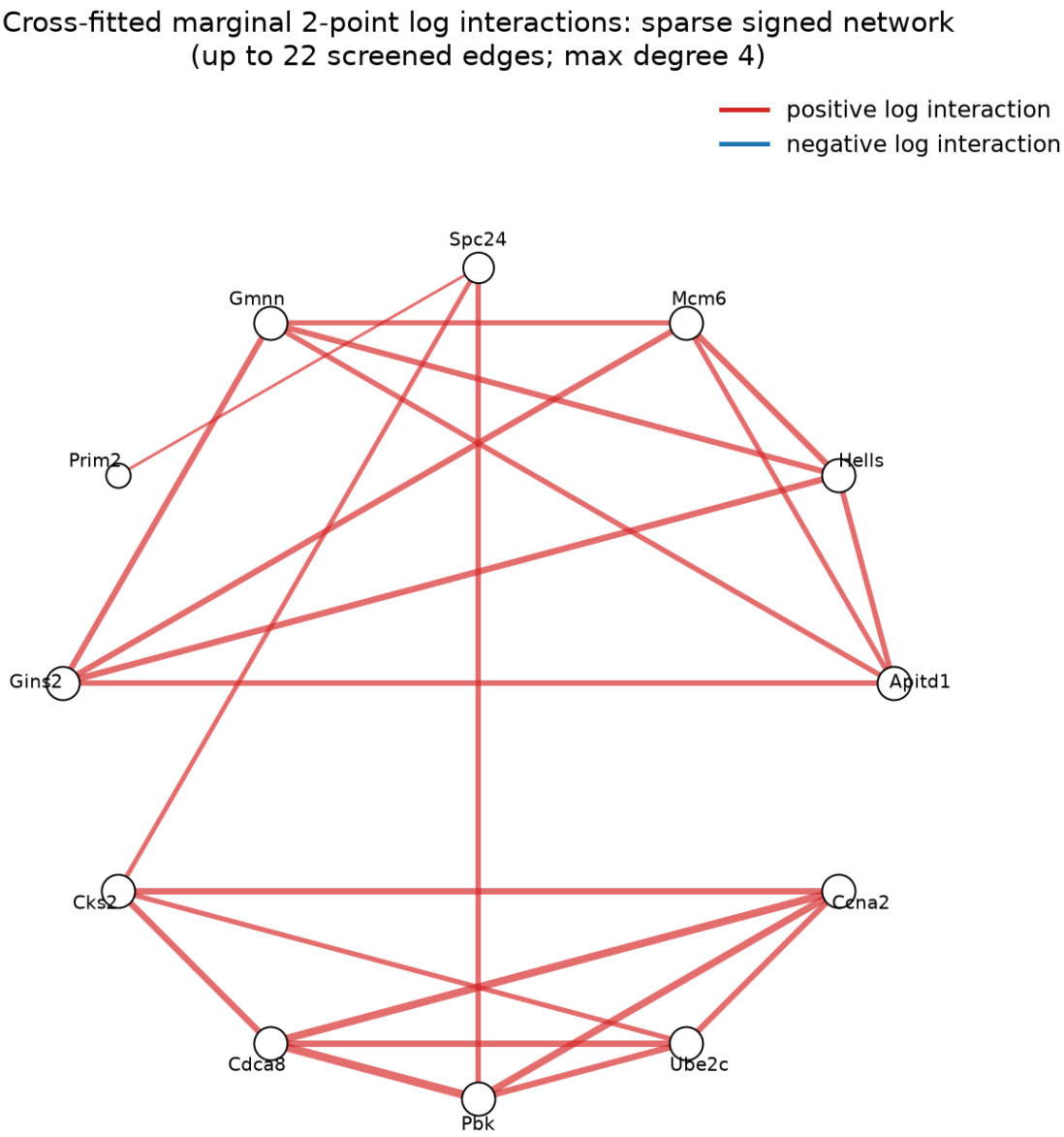


Figure 4: 12-gene marginal interactions, order 2: network.

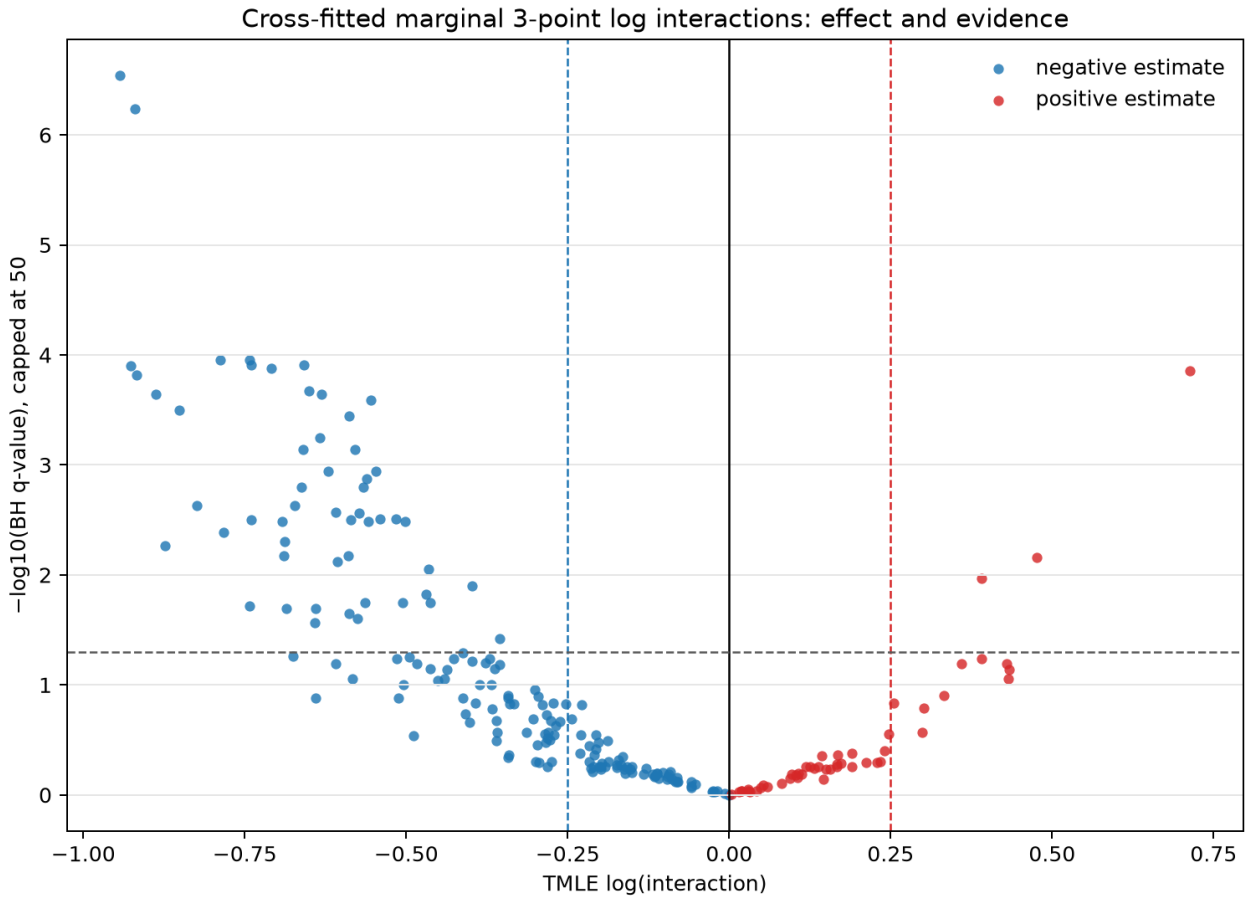


Figure 5: 12-gene marginal interactions, order 3: overview.

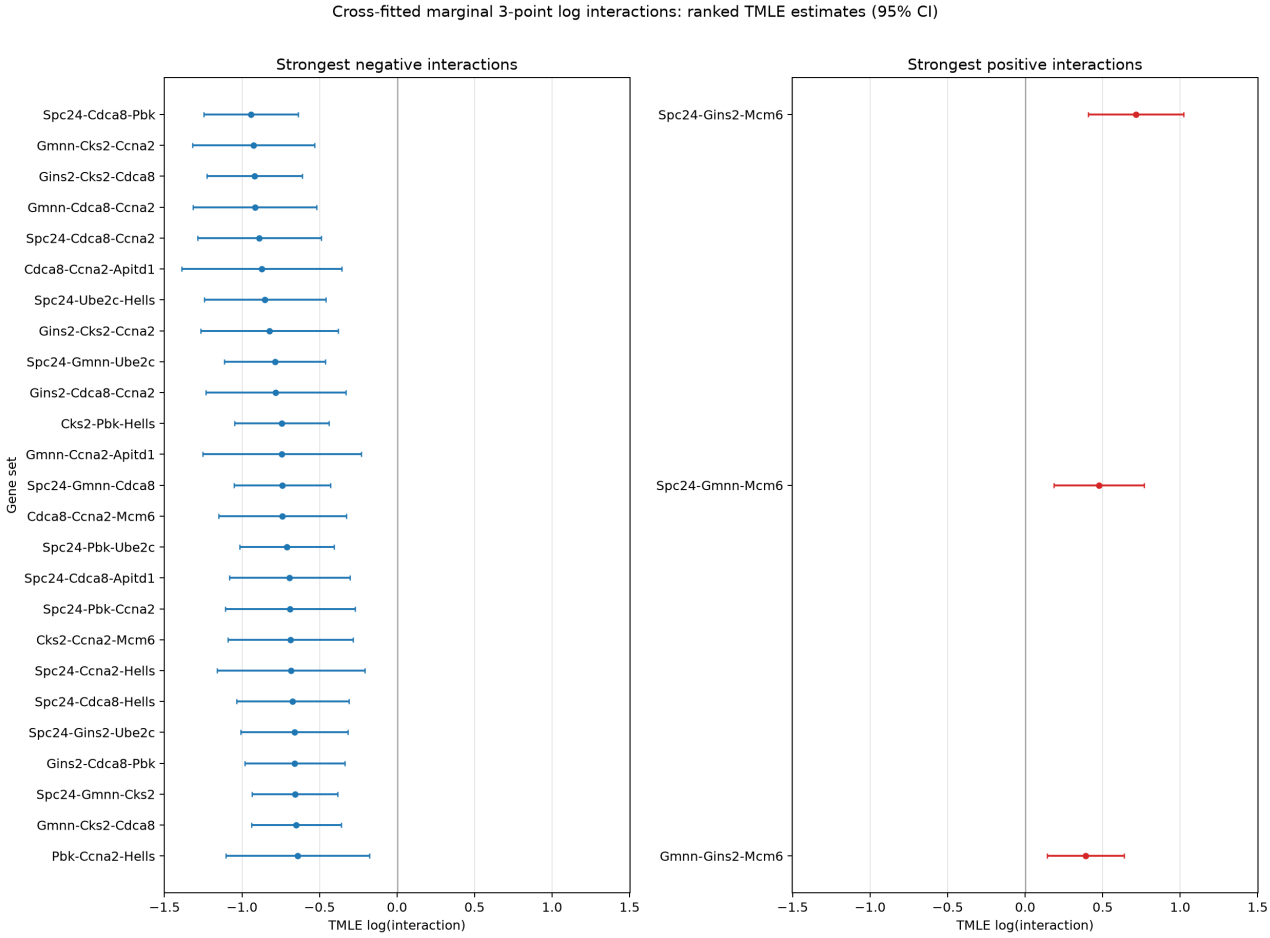


Figure 6: 12-gene marginal interactions, order 3: forest summary.

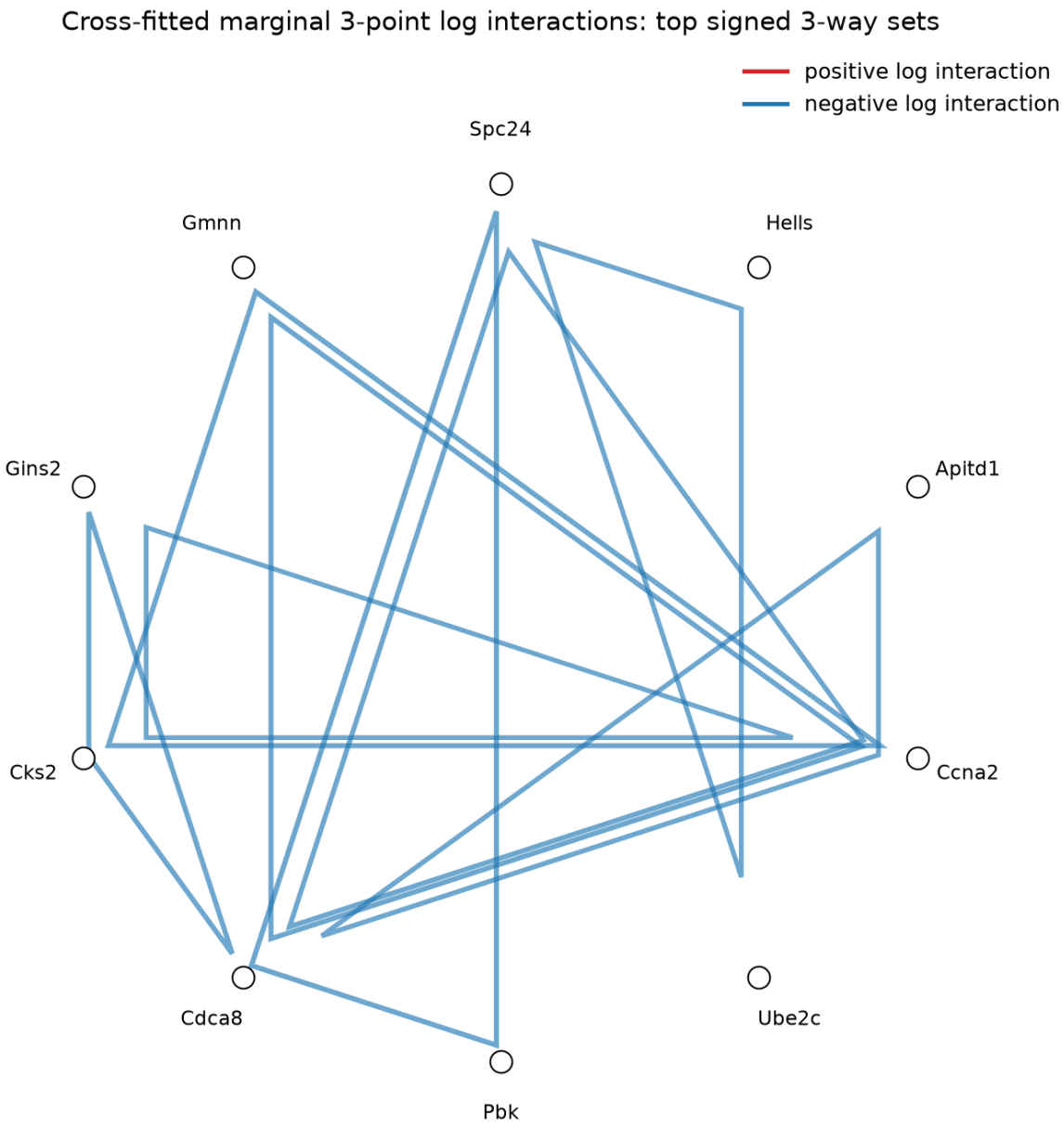


Figure 7: 12-gene marginal interactions, order 3: hypergraph.

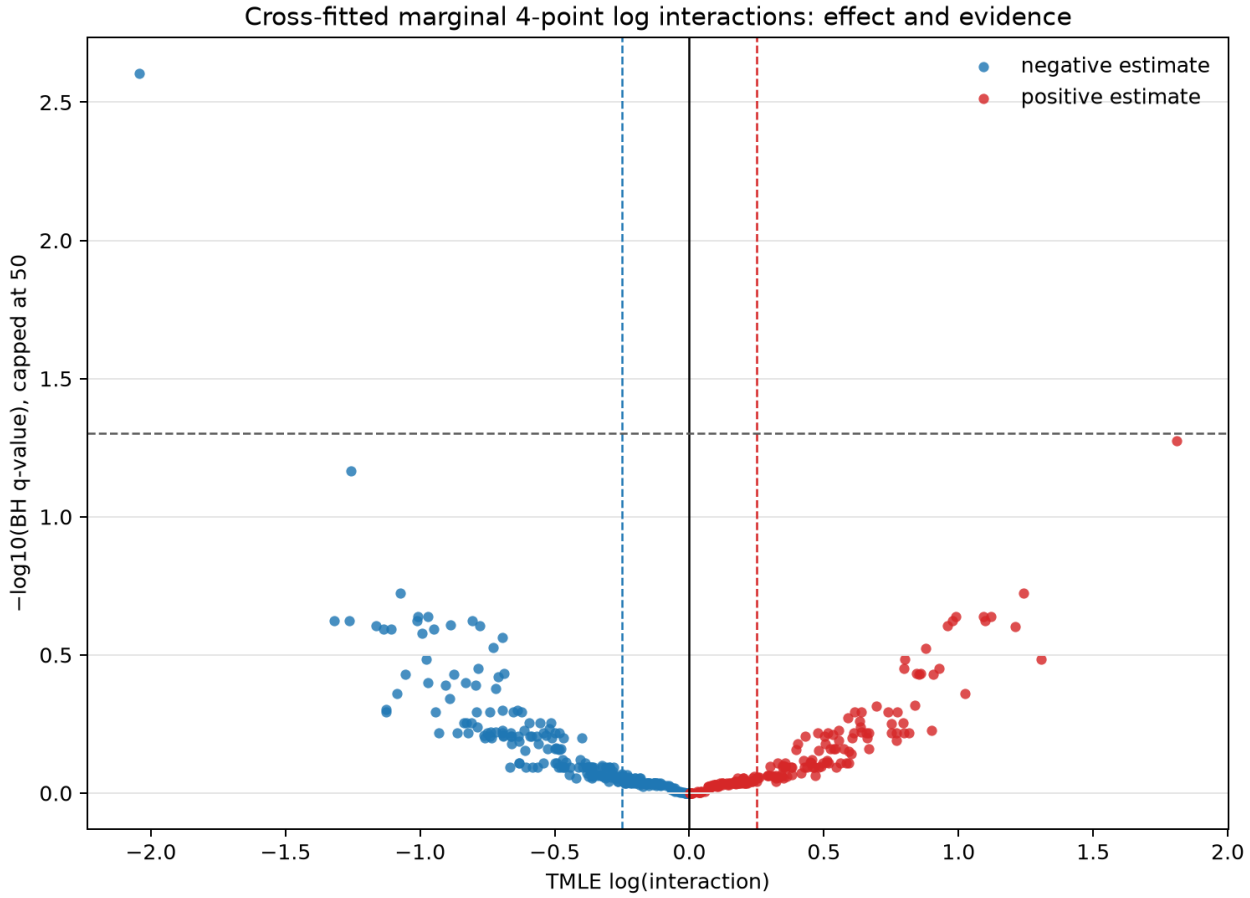


Figure 8: 12-gene marginal interactions, order 4: overview.

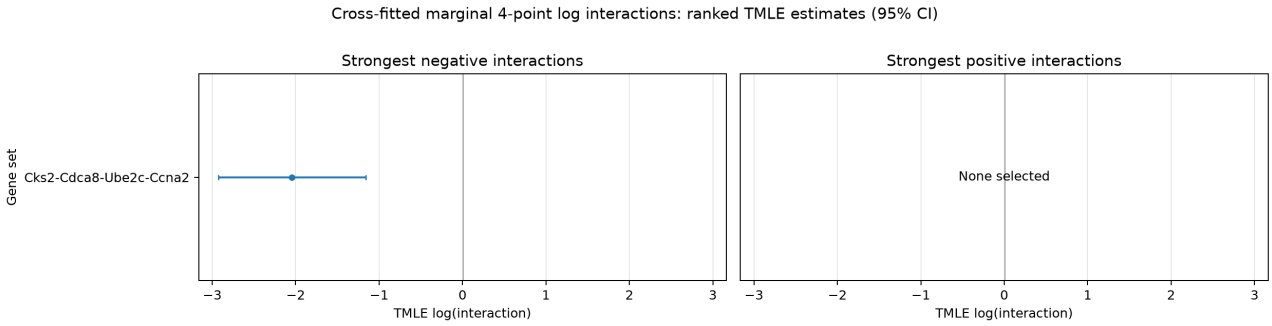


Figure 9: 12-gene marginal interactions, order 4: forest summary.

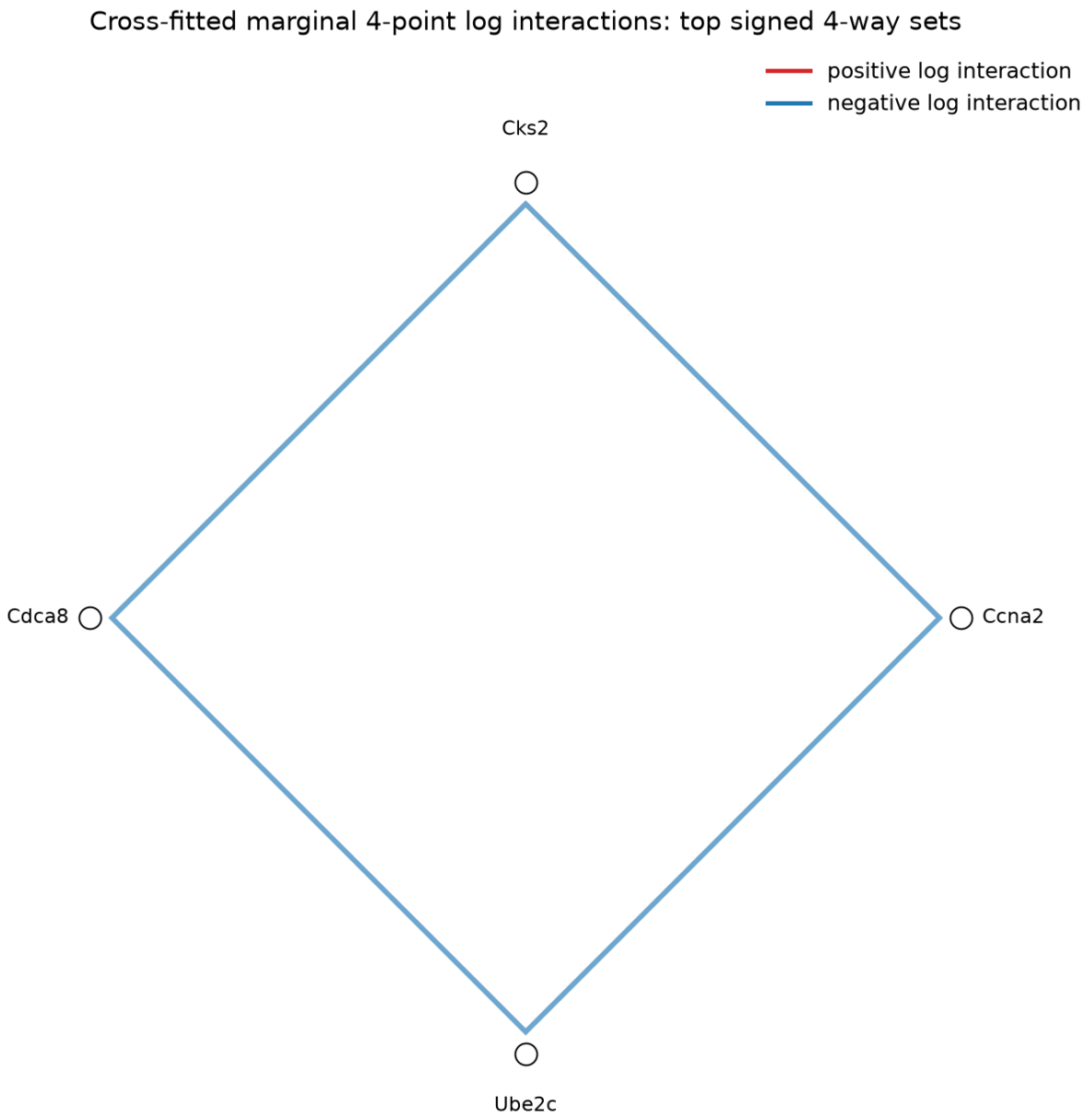


Figure 10: 12-gene marginal interactions, order 4: hypergraph.

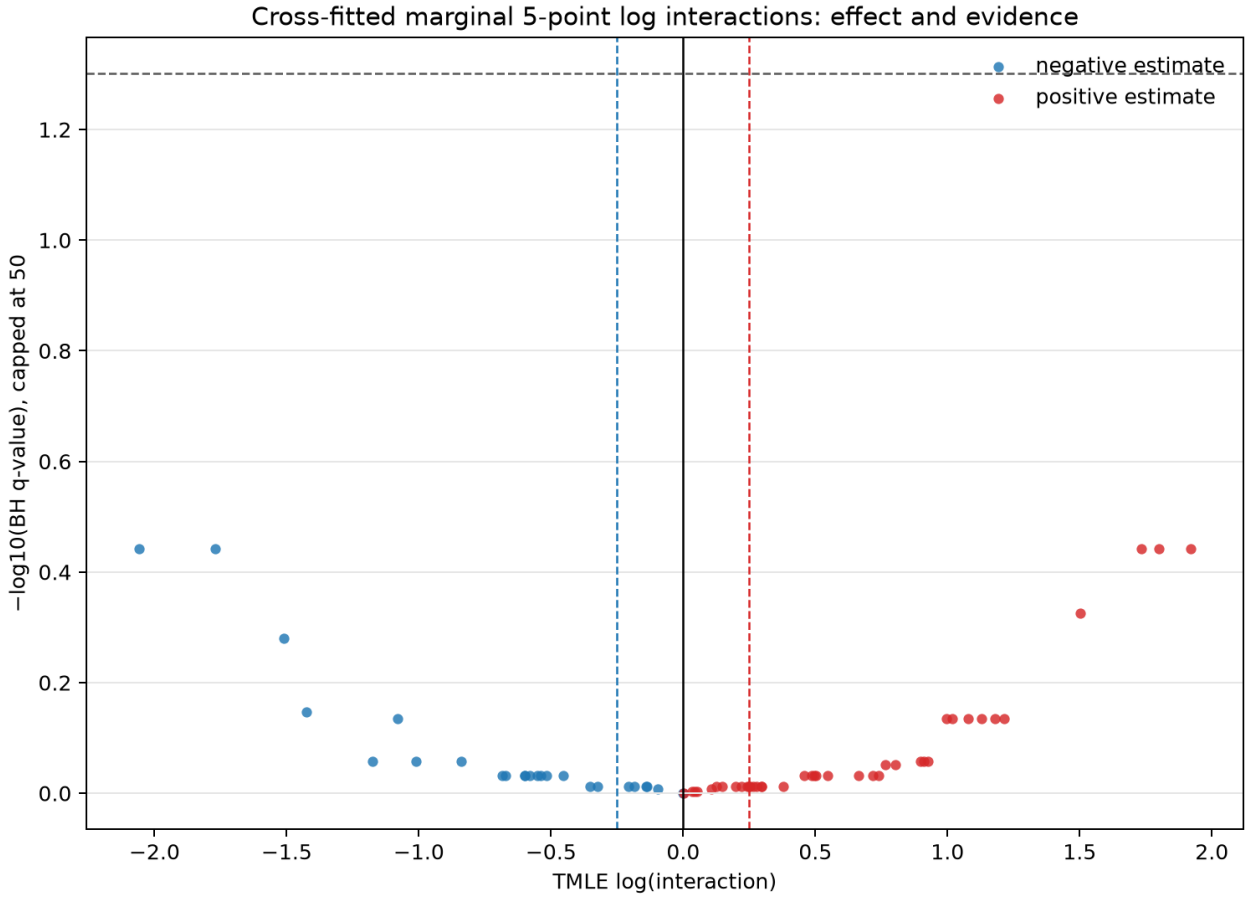


Figure 11: 12-gene marginal interactions, order 5: overview.

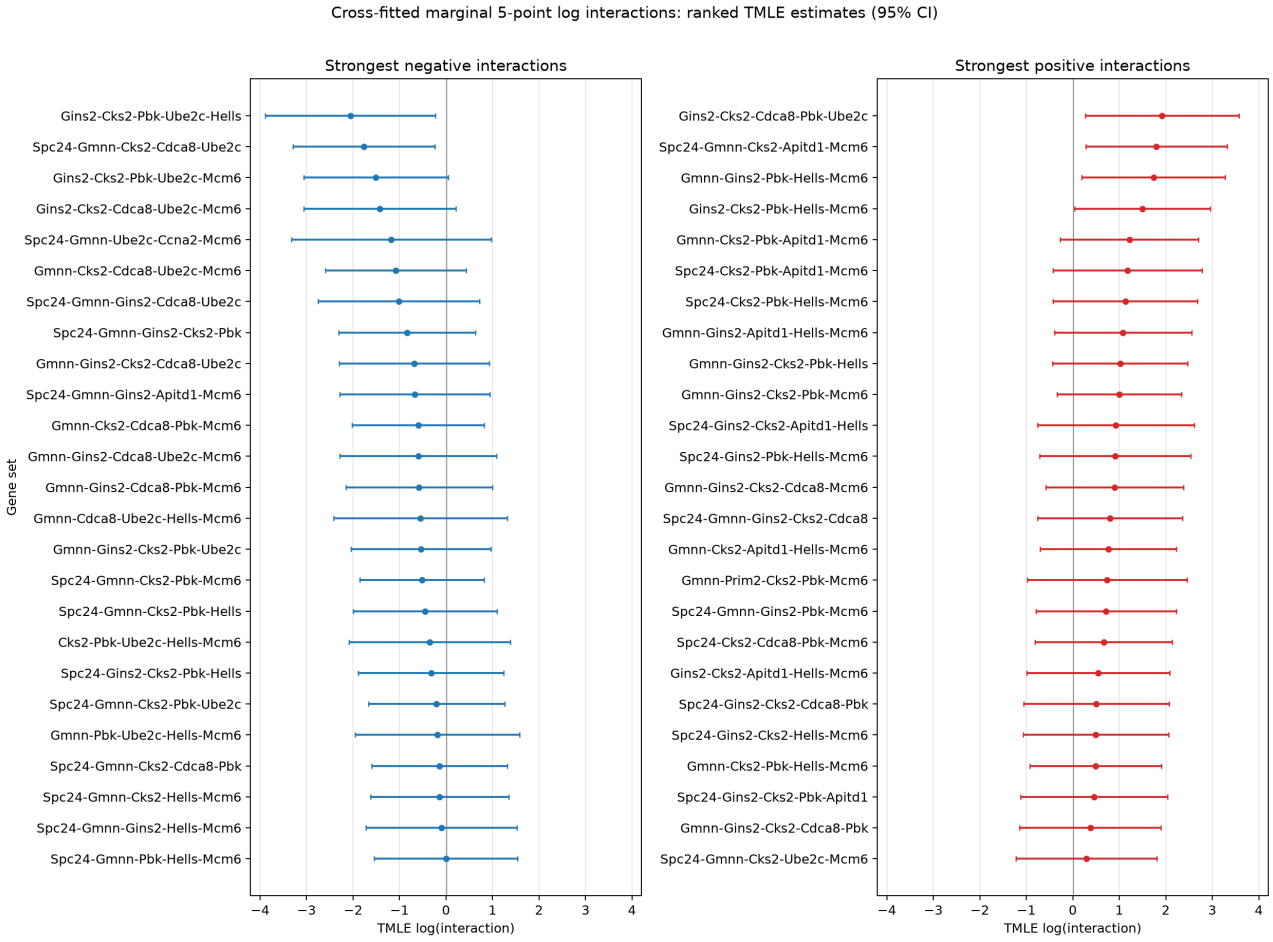


Figure 12: 12-gene marginal interactions, order 5: forest summary.

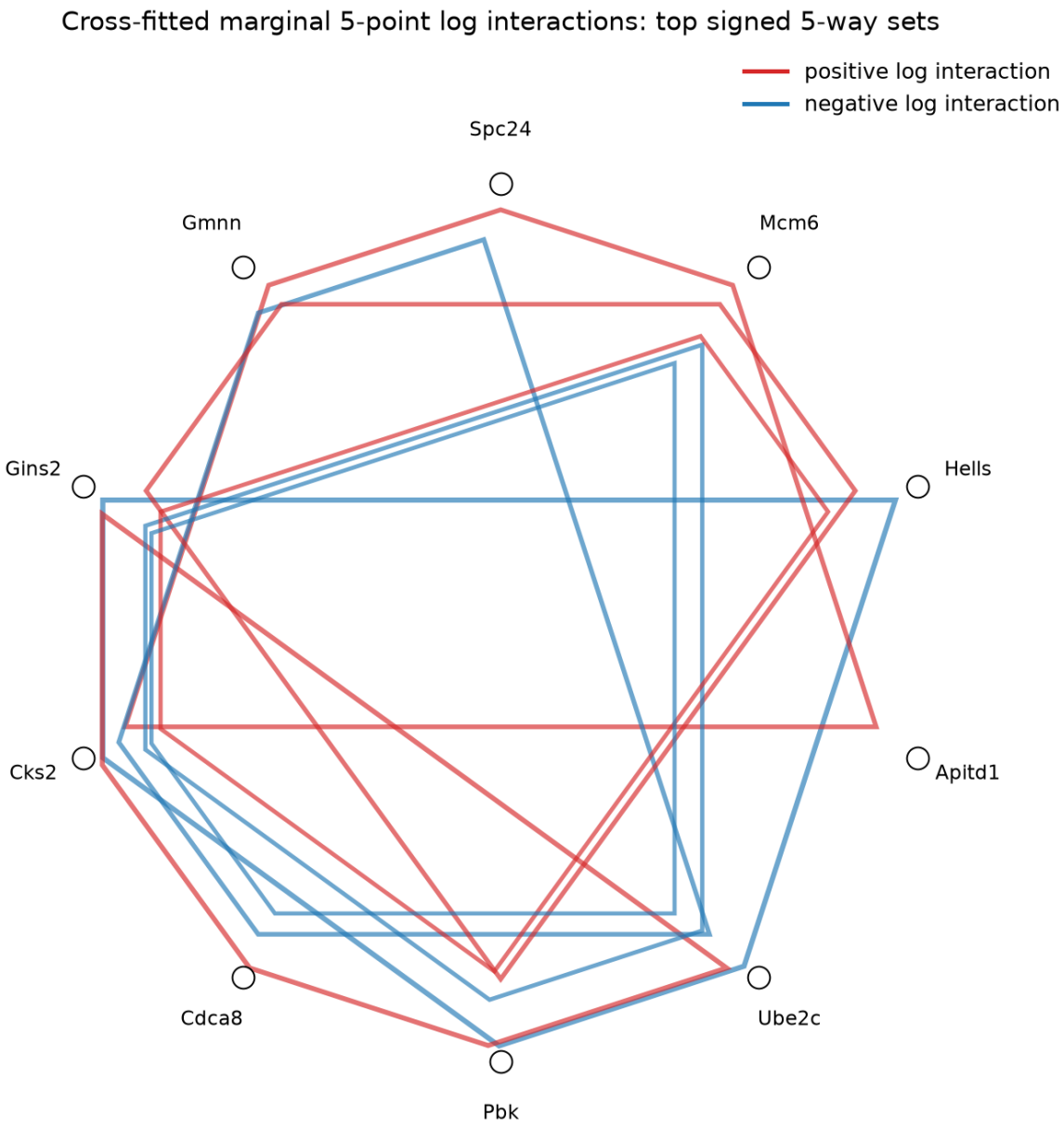


Figure 13: 12-gene marginal interactions, order 5: hypergraph.

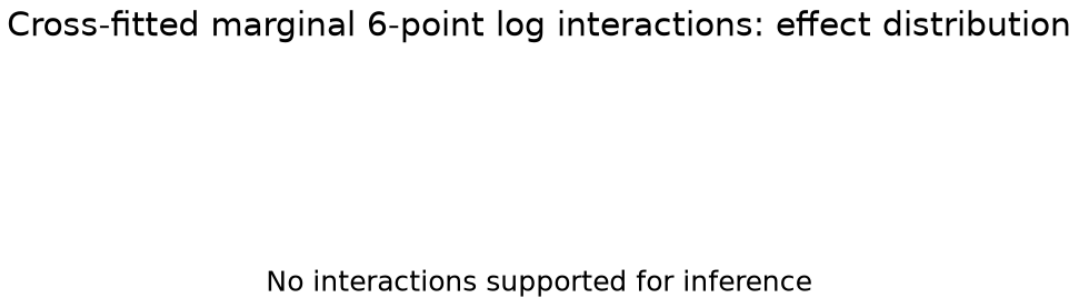


Figure 14: 12-gene marginal interactions, order 6: overview.

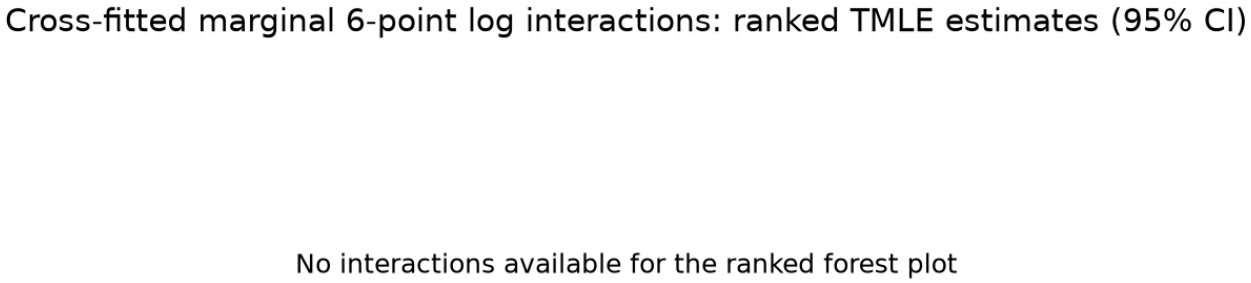


Figure 15: 12-gene marginal interactions, order 6: forest summary.

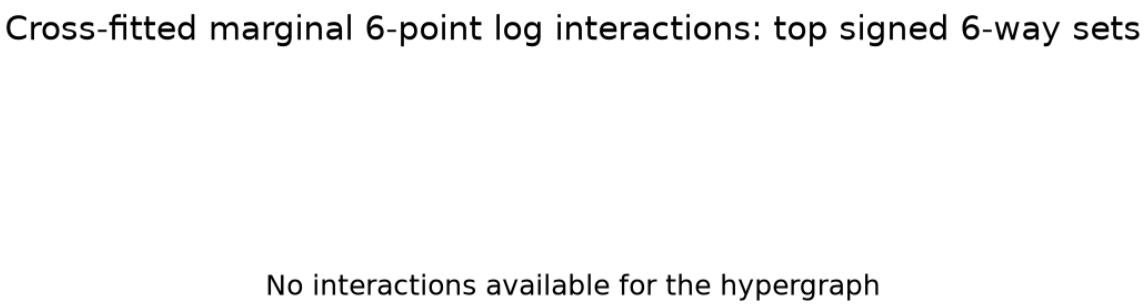


Figure 16: 12-gene marginal interactions, order 6: hypergraph.

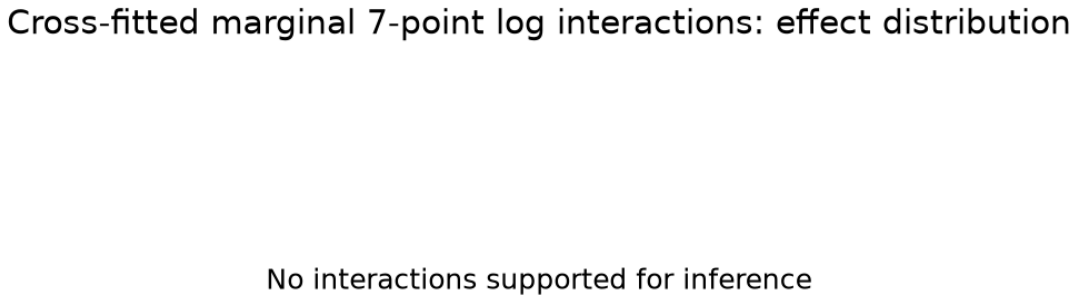


Figure 17: 12-gene marginal interactions, order 7: overview.

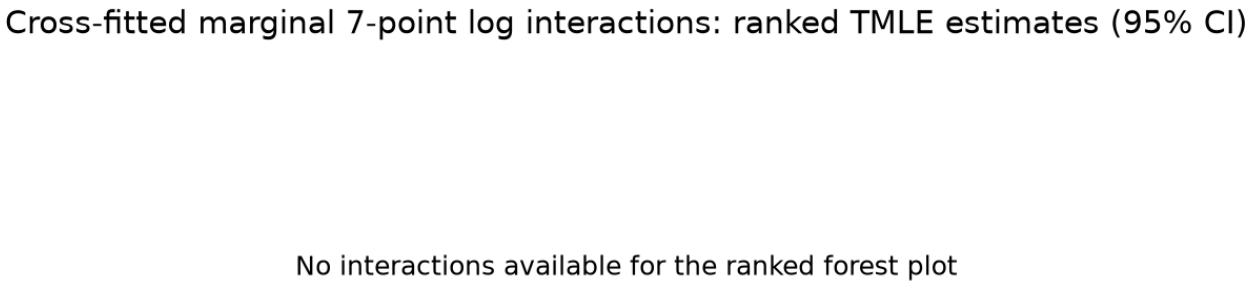


Figure 18: 12-gene marginal interactions, order 7: forest summary.

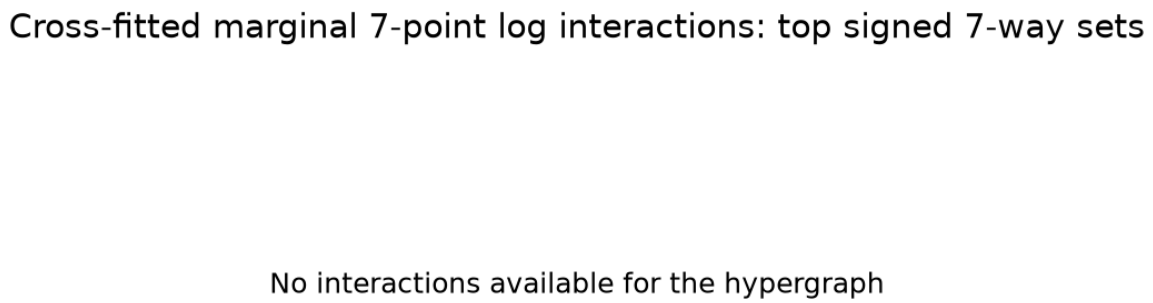


Figure 19: 12-gene marginal interactions, order 7: hypergraph.

6 Results: 999-gene study

The all-pairs run constructs 498,501 four-cell tables from its exact cache. The fresh sequential pipeline took 15.66 s: 12.16 s fresh training plus 3.50 s for a forced cache rebuild, all-pairs TMLE, results, and figures. With `minimum-cell-count=0` and `minimum-probability=1e-12`, 389,779 pairs were finite, targeted, and supported; 108,722 did not have a finite targeted estimate in both folds. BH was therefore calculated over 389,779 supported pairs; 132,177 pass both the default $q \leq 0.05$ and $|\theta| \geq 0.25$ display cuts. Higher-order 999-gene results are explicitly exploratory because pairwise results select their candidates; exhaustive scans beyond pairs are combinatorially infeasible.

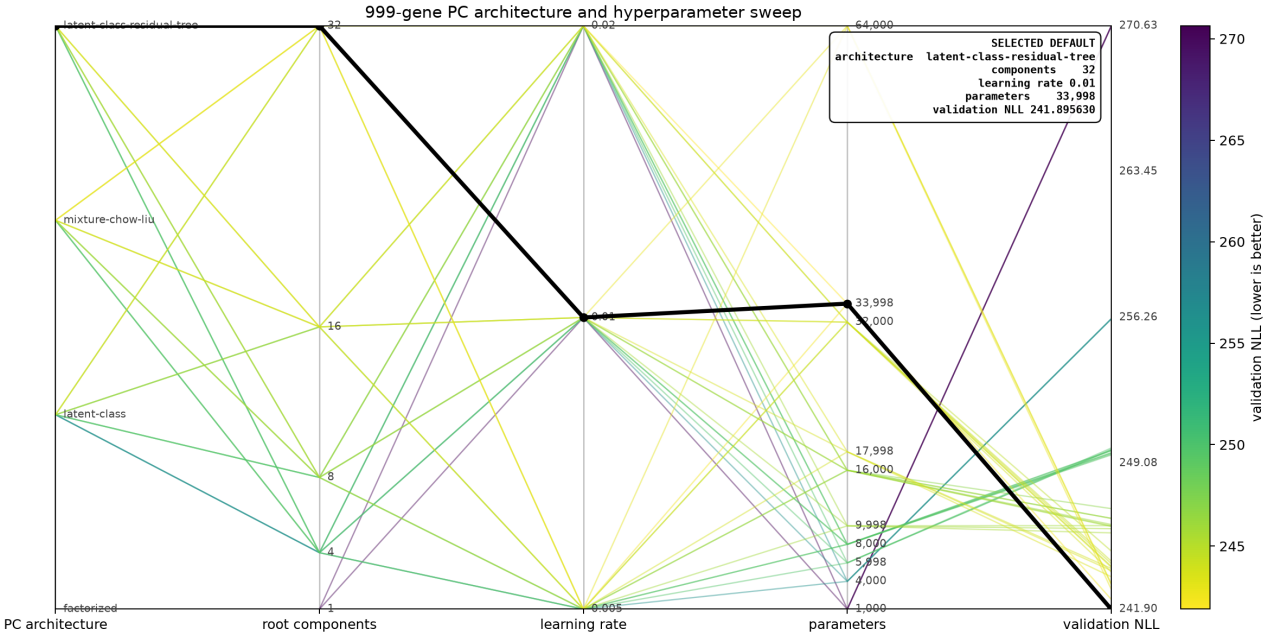


Figure 20: 999-gene PC architecture sweep retained for provenance.

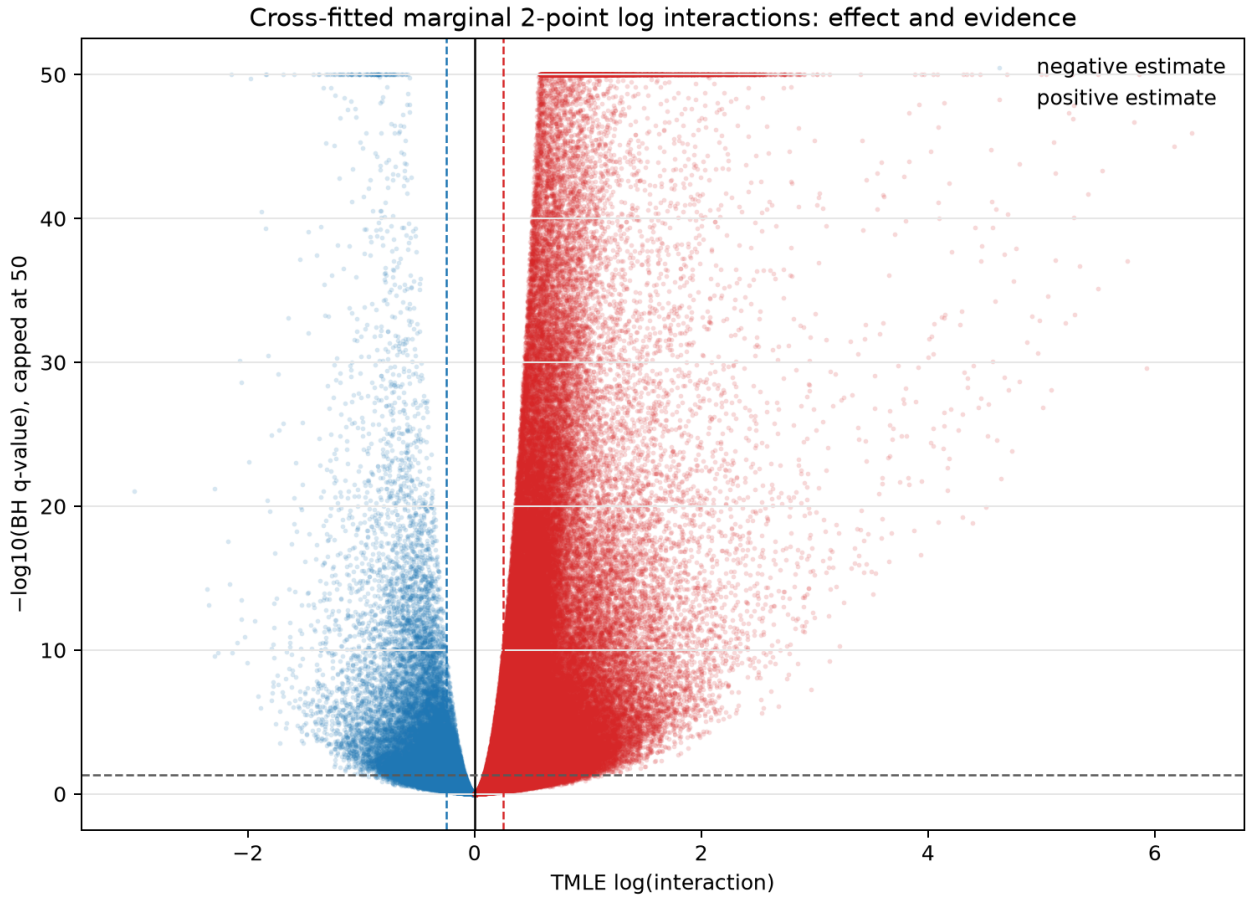


Figure 21: 999-gene marginal pairwise interactions: overview.

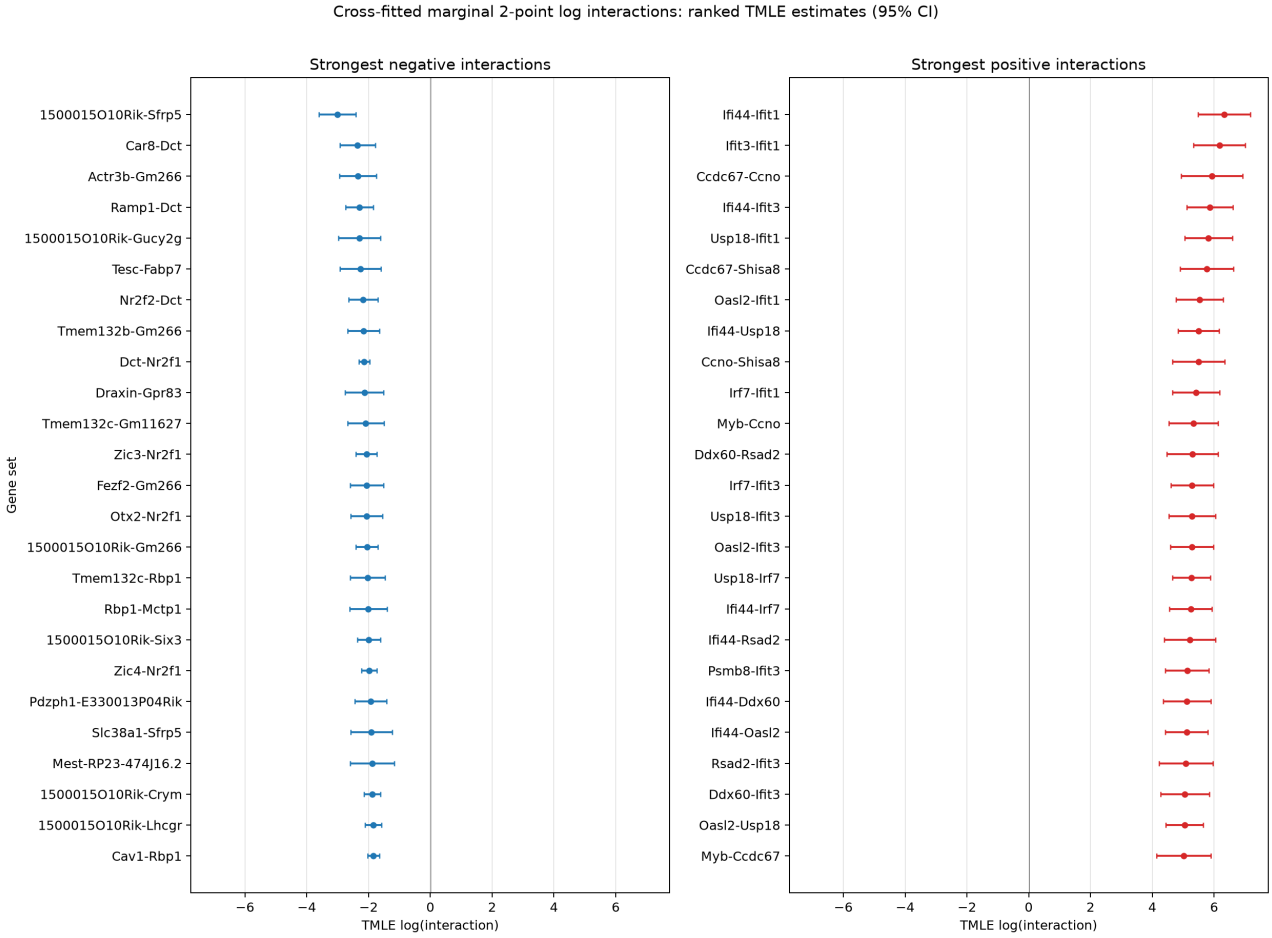


Figure 22: 999-gene marginal pairwise interactions: forest summary.

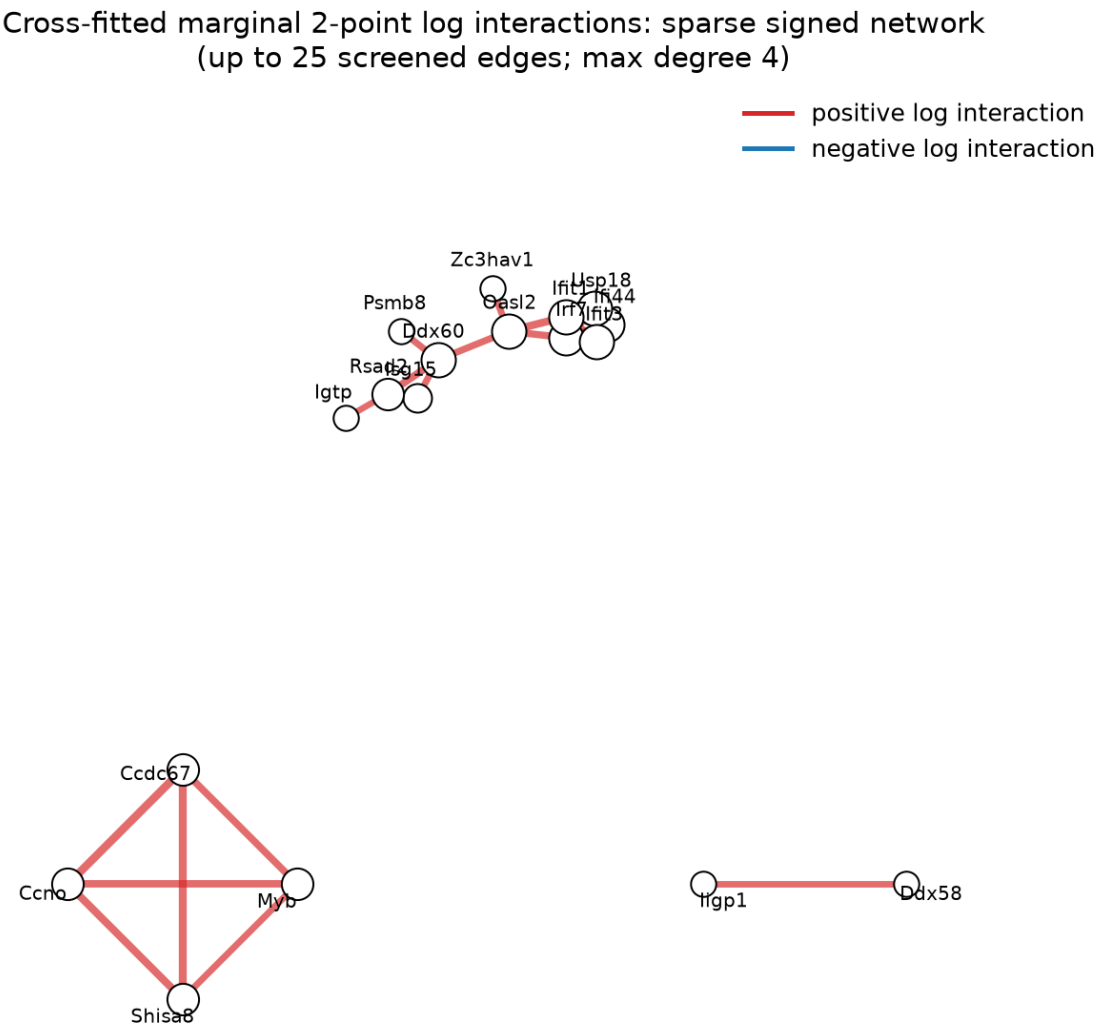


Figure 23: 999-gene marginal pairwise interactions: sparse signed network.

A Complete 12-gene analysis guide (verbatim)

```
# 12-gene marginal-estimand analysis

This guide explains the current analysis of a fixed panel of 12 genes. It uses the shared probabilistic-circuit (PC)
↪  models and batched TMLE code. Files in `legacy_code/` are kept only for historical reference; they did not create
↪  the current results.

## At a glance

The workflow does four things:

1. selects a PC architecture using validation likelihood only;
2. trains two PCs on opposite halves of the cells, so each cell is evaluated by a model that did not see it during
↪  training;
3. estimates the association for every requested gene set; and
4. reports uncertainty only when every needed cell has enough raw data and the fitted model assigns it a usable
↪  probability.

An interaction here is an association in binary gene-expression states. It is not evidence that one gene directly
↪  regulates another, and it is not a causal effect.

## Data, panel, and saved files

The panel has a fixed order. That order is part of the analysis and must not change.

~~~text
Spc24  Gmnn  Prim2  Gins2  Cks2  Cdca8
Pbk    Ube2c Ccna2  Apitd1 Hells Mcm6
~~~

The source CSV contains 11,950 cells and 1,000 numeric columns. Its unnamed first column is constant metadata, so the
↪  analysis removes it. The remaining 999 named columns must contain binary values (0 or 1). The panel loader takes
↪  the 12 names above, in the displayed order. It does not filter cells or genes: all 11,950 cells are used.

`--max-samples` is only for development. It changes the data fingerprint, so a reduced-data run cannot accidentally
↪  reuse a full-data checkpoint.

Every checkpoint and result file stores a SHA-256 fingerprint of the ordered gene names and binary values. Loading
↪  fails if the input file has changed, its columns have been reordered, or a different panel is supplied.

| File | What it contains |
| --- | --- |
| `output/pc_architecture_sweep_12_genes.csv` | Results for every model-selection candidate. |
| `output/pc_architecture_selected_config_12_genes.json` | The settings selected by validation likelihood. |
| `output/cross_fitted_pcs_12_genes.pt` | The two fitted fold-specific PCs and run provenance. |
| `output/{2,3,4,5,6,7}_pt_marginal_estimand_interactions_12_genes.pt` | Machine-readable marginal-interaction
↪  results for one order. |
| `plots/{order}_pt_marginal_estimand_interactions_12_genes.png` | A labelled marginal-interaction plot for every
↪  requested gene set. |
| `plots/*_overview.png`, `*_forest.png`, `*_network.png`, `*_hypergraph.png` | Smaller summary figures. |
| `logs/*.txt` | Plain-text run details, support checks, and results. |

Run the usual workflow from the repository root:

~~~bash
python sweep_pc_architectures.py --analysis 12
python train_cross_fitted_pcs.py --analysis 12
python cross_fitted_interactions.py --analysis 12 --interaction-order 2
~~~

## 1. Choose the PC model

Model selection is separate from interaction estimation. With `seed=0`, the 11,950 cells are randomly split into
↪  9,560 selection-training cells and 2,390 validation cells (80/20). Each candidate learns from the first group and
↪  is ranked by its mean validation negative log likelihood (NLL) on the second. Lower NLL is better.

The 20-epoch selection run evaluates 39 candidates:

- four architectures: factorized, latent-class, mixture-Chow--Liu, and latent-class-residual-tree;
- 4, 8, 16, or 32 components for each non-factorized architecture (the factorized baseline has one component);
- AdamW learning rates of 0.005, 0.01, or 0.02; and
```

- a batch size of 1,024 cells, weight decay of `1e-5`, and seed 0.

For tree-based candidates, a Chow--Liu tree is learned once using only the selection-training cells. It uses smoothed pairwise mutual information, chooses the maximum-spanning tree, and roots it at the gene with highest entropy. The validation cells play no part in learning this tree.

The saved winner was:

Selected setting	Value
Architecture	mixture-Chow--Liu
Mixture components	32
AdamW learning rate	0.02
Weight decay	`1e-5`
Validation NLL	4.081627
Trainable parameters	832

No interaction estimate, p-value, or q-value was used to select these settings.

What the selected PC represents

The selected PC is a weighted mixture of 32 binary tree distributions:

~~~text
p(x) = sum_z pi_z p(x_root | z) product_{j != root} p(x_j | x_parent(j), z)
~~~

In plain language, each mixture component represents a broad expression pattern. Within a component, the genes follow the same tree layout, but their probabilities may differ. The component weights `pi_z` are normalized with softmax. Each component has its own root probability and two conditional probabilities for every non-root gene (one for each parent state). Starting values come from smoothed empirical probabilities with small noise, keeping probabilities away from exactly zero and one.

2. Train two fold-specific models

The selection sweep chooses the architecture and a few hyperparameters. It is not the final training record. The saved checkpoint was trained with these settings:

Checkpoint setting	Recorded value
PC / components	mixture-Chow--Liu / 32
Cross-fitting folds	2
Training / held-out cells per fold	5,975 / 5,975
Epochs	20
Training minibatch	128 cells
AdamW learning rate / weight decay	0.02 / `1e-5`
Fold seeds	0 and 1
Device	CPU
Training time	1.19 s model-fit/save timer; 2.13 s complete fresh training command
Final training NLL, folds 1 and 2	4.024783, 4.044821

During every epoch, the training cells are shuffled and AdamW minimizes the average NLL. A training batch always means a number of **cells**; it never means a group of the 12 genes.

Cross-fitting works as follows:

~~~text
All 11,950 cells -- seeded split --> F1 (5,975) + F2 (5,975)

Train PC_1 on F2  --> estimate and target results for F1
Train PC_2 on F1  --> estimate and target results for F2

Combine the two fold results, weighting each by its held-out sample size.
~~~

This protects against evaluating a cell with a model that memorized it: every cell is evaluated by a PC trained on the other fold. The two held-out folds cover every cell exactly once.

Recorded fresh end-to-end runtimes

The following are complete wall-clock timings from a fresh full-panel run on 19 October 2026. It trained the two 20-epoch fold PCs from the source CSV before any interaction work; no checkpoint, interaction result, or cache from an earlier run was read. Each interaction command then read the checkpoint produced moments earlier in that same master run. The run used `minimum-cell-count=1`, `minimum-probability=1e-12`, `batch-size=512`, 64 TMLE bisection iterations, default plots, and no reduced data.

Order	Full-panel targets	Fresh PC training (once)	Full interaction command	Fresh fit + one order
2-point	66	2.13 s	1.60 s	3.73 s
3-point	220	2.13 s	2.01 s	4.14 s
4-point	495	2.13 s	2.95 s	5.08 s
5-point	792	2.13 s	4.47 s	6.60 s
6-point	924	2.13 s	5.28 s	7.41 s
7-point	792	2.13 s	4.82 s	6.95 s

The table reports the `real` time from `/usr/bin/time -p` around each whole Python command, so it includes interpreter startup and imports, loading the CSV and freshly written checkpoint, querying every gene set, exact PC probabilities and held-out counts, TMLE plus SE/p-value/BH calculation, result/report saving, and rendering the tall full-panel figure plus overview, forest, and network/hypergraph figures. The per-order marginal-estimand logs also retain their smaller internal analysis timer; the table deliberately uses the complete command timer rather than assigning unsupported substep timings. Training once and then running all six orders took 23.26 s by summing the recorded command times (`2.13 + 1.60 + 2.01 + 2.95 + 4.47 + 5.28 + 4.82`), rather than fitting six separate sets of PCs.

3. Calculate model probabilities and observed counts

For interaction order `r`, the default run examines every `C(12,r)` gene set:

Order	Sets evaluated
2 (pairs)	66
3 (triples)	220
4	495
5	792
6	924
7	792

Use `--genes` when you want one named set instead of the full panel.

For each gene set `S`, the PC calculates the probability of every binary pattern `a` in `{0,1}^r`. For example, a pair has four patterns: 00, 01, 10, and 11. These are exact marginal probabilities under the fitted PC, not probabilities estimated by simulating cells. The tree PC uses batched message passing: it fixes the selected gene states, sums over the remaining tree nodes, then averages across mixture components.

The interaction command's `--batch-size` is a number of **gene sets** processed together. It is not a count of genes or cells.

For the held-out cells, the code counts every pattern for every requested set in one batch. It encodes each pattern as a binary integer and uses `torch.bincount`, rather than looping over cells in Python.

4. Interaction estimate, TMLE, and uncertainty

For a set `S` of order `r`, the reported interaction is:

~~~text
theta(p) = sum_a s(a) log p(X_S=a)
s(a) = (-1)^(r - sum(a))
~~~

For a pair, this simplifies to:

~~~text
log[p00 p11 / (p01 p10)]
~~~

That is the log odds ratio. Zero means no interaction on this scale; positive and negative values indicate positive and negative association. For sets larger than pairs, exponentiating the result gives an alternating cross-product ratio. It is not a correlation and it is not a causal effect.

What TMLE changes

For each fold and target, TMLE starts from the PC probability table `p_0` and makes a small, targeted update:

~~~text
p_epsilon(a) = p_0(a) + epsilon s(a)
~~~

`epsilon` is restricted so that every updated probability remains strictly positive. The code moves the endpoints of that valid interval slightly inward, then uses 64 vectorized bisection steps to solve:

~~~text
sum_a p_n(a) s(a) / p_epsilon(a) = 0
~~~

Here, `p_n` is the observed distribution in the held-out fold. This equation says that the empirical efficient-influence-function (EIF) mean is zero. The EIF is **not** minimized.

At the updated table `p_star`, the centered EIF is:

~~~text
H(a) = s(a)/p_star(a) - E_pstar[s(A)/p_star(A)]
~~~

The two fold estimates are combined using their held-out sample sizes. The pooled EIF values give the standard error (SE):

~~~text
SE = sqrt([sum H^2 - (sum H)^2/N] / [(N-1) N])
~~~

The result table then gives a 95% Wald interval (`theta +/- 1.959964 SE`), a two-sided normal p-value for the null value zero, and a Benjamini--Hochberg (BH) q-value.

When a result is eligible for inference

The table uses two deliberately different statuses:

- **Targeted:** The initial model table is finite and strictly positive, and the bisection equation has a valid solution in both folds. This produces a finite point estimate.
- **Supported for unregularized inference:** The target is targeted and, in each fold, every one of its `2^r` observed patterns has at least `--minimum-cell-count` cells, every initial model probability is at least `--minimum-probability`, and the table is finite.

Only supported results receive an SE, confidence interval, p-value, and q-value. A targeted but unsupported result can still be useful as a descriptive point estimate, but it is intentionally not used for unregularized statistical inference.

For the freshly saved pairwise panel result, `minimum-cell-count=1` and `minimum-probability=1e-12`; all 66 pairs are supported. At larger interaction orders, sparse pattern counts--not computation--usually become the main scientific limit.

> **What "supported unregularized TMLE" means here.** The raw held-out counts are used exactly as observed. The code does not add pseudocounts, a continuity correction, a probability floor, a ridge penalty, or variance regularization to repair a rare or empty pattern. For a pair in this saved run, each of 00, 01, 10, and 11 must appear at least once in F1 and at least once in F2. The fitted PC must also assign each pattern probability at least `1e-12` in both folds. These thresholds only decide whether inference is reported; they do not change counts or probabilities.
>
> A result may be targeted without being supported. For example, the score equation may have a finite solution even though one pattern is too rare for the prespecified inference rule. Such a result has no SE, CI, p-value, or q-value. If no strictly positive bisection solution exists, it is not targeted either.

What the BH q-value in an overview plot means

BH is applied **only** to the supported targets in the result being plotted; unsupported targets have no p-value or q-value and are not counted. Let `m` be that number of supported targets, let `p_h` be the two-sided Wald p-value for target `h` under its null hypothesis of zero log interaction, and write the ordered p-values as

~~~text
p_(1) <= p_(2) <= ... <= p_(m).
~~~

For a chosen false-discovery-rate (FDR) level `alpha`, the BH procedure finds the largest rank `k` for which

~~~text
p_(k) <= k * alpha / m,
~~~

~~~

then calls ranks `1, ..., k` discoveries. The reported value for the target at rank `i` is the corresponding monotone BH-adjusted p-value:

~~~text
q_(i) = min_{j=i,...,m} min(1, m * p_(j) / j).
~~~

The value is then returned to its original target. Thus `q_h <= alpha` means exactly that target `h` belongs to the BH discovery set if BH is run at level `alpha`; equivalently, `q_h` is the smallest BH FDR level at which it would be selected. The name "q-value" here means this BH-adjusted p-value, not a posterior probability or a local false-discovery rate.

In the overview plot, a point for target `h` has vertical coordinate

~~~text
y_h = -log10(q_h),
~~~

with values above 50 visually capped at 50. Higher points have smaller BH-adjusted p-values: `q=0.05`, `0.01`, and `0.001` map to about `1.301`, `2`, and `3`, respectively. The horizontal line is therefore at `-log10(0.05) = 1.301`; points on or above it meet the BH `q <= 0.05` criterion. The x-coordinate and colour describe the estimated TMLE log interaction and its sign, respectively. They do not change the q-value, which comes from a two-sided p-value.

BH controls the FDR of the *set* selected at a fixed cutoff, where

~~~text
FDP = V / max(R, 1),       FDR = E[FDP],
~~~

`R` is the number of BH discoveries and `V` is the (unobserved) number of those discoveries whose null hypotheses are true. Under valid null p-values and the usual BH dependence conditions (independence, or positive regression dependence on the true nulls), selecting every target with `q <= 0.05` gives `FDR <= 0.05`. It does **not** say that five percent of a particular subset, one plotted point, or one named gene set is false, and it does not establish causation. The targets share cells and genes, so their tests are dependent; the standard BH guarantee is not a guarantee under arbitrary dependence. A confirmatory analysis needing protection under arbitrary dependence would need a different prespecified correction, such as Benjamini--Yekutieli.

References: Benjamini and Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B* 57(1), 289--300 (1995), [doi:10.1111/j.2517-6161.1995.tb02031.x](https://doi.org/10.1111/j.2517-6161.1995.tb02031.x); Benjamini and Yekutieli, "The Control of the False Discovery Rate in Multiple Testing under Dependency," *Annals of Statistics* 29(4), 1165--1188 (2001), [doi:10.1214/aos/1013699998](https://doi.org/10.1214/aos/1013699998).

`empirical_log_interaction` is included only as a full-data descriptive comparison. It uses all 11,950 cells, is `NaN` if any observed pattern is empty, is not cross-fitted, and is not used for TMLE inference.

5. Read the result table and plots

The `.pt` result table includes the gene set, PC plug-in effect, TMLE effect, SE, confidence interval, p-value, q-value, smallest observed count, smallest model probability, and targeted/supported flags. For pairs, exponentiate the log effect to obtain the odds ratio. Do not treat a screened result as proof of direct regulation or causation.

Full-panel marginal comparison: `{order}_pt_marginal_estimand_interactions_12_genes.png`

This tall figure intentionally gives every requested set its own row.

- Rows are ordered from the largest finite empirical log interaction to the smallest; sets with undefined empirical values appear last.
- A circle shows the PC plug-in estimate.
- An orange x with a bar shows a supported TMLE estimate and its 95% CI. A finite, unsupported TMLE estimate is an orange x without a CI.
- A green star shows the full-data empirical comparison when it is defined.
- An orange line connects the plug-in and finite TMLE estimates.
- The dashed vertical line at zero marks no log interaction.

Overview, forest, network, and hypergraph figures

These smaller figures begin with results that are supported and have finite TMLE estimates and confidence intervals. With the default settings, their display set is
~~~

~~~text
D = {h : supported, finite estimate/CI, q_h <= 0.05,
          and |TMLE log interaction_h| >= 0.25}.
~~~

The two cuts serve different purposes: `q <= 0.05` is the multiple-testing evidence screen, whereas `|log interaction| >= 0.25` is an effect-size screen (an odds ratio at least `exp(0.25) ≈ 1.28`, or at most `exp(-0.25) ≈ 0.78`, for pairs). If `D` is empty, the forest, pair network, and hypergraph fall back to all supported finite results so that the figure is not blank. Consequently, a fallback figure is a ranking of supported results, not a set that passed both default cuts. This is a display and ranking rule; the full result table remains the complete record. The overview is different: it always shows every supported result.

- **Overview:** Shows every supported effect against `-log10(BH q-value)`, capped at 50 only for drawing. Red means positive and blue means negative. Dashed vertical lines mark +/-0.25; the horizontal line at 1.301 marks `q=0.05`. The BH family is all supported targets in that plotted result, not only points passing the effect-size display cut.
- **Forest:** Separates `D` into negative estimates (`theta < 0`) and non-negative estimates (`theta >= 0`). Within each sign it orders by the estimate itself--most negative first on the left and most positive first on the right--and takes `min(25, number available)`. Thus the two panels are selected independently, not balanced to contain 25 each. A panel has fewer than 25 points when fewer than 25 display-set results have that sign (or, after the no-hit fallback, fewer than 25 supported finite results have that sign); it says nothing about a 25th result being removed for a confidence interval or an additional significance test. A panel with no eligible results says "None selected." Points are TMLE log effects, bars are 95% CIs, and the black line is zero. The default 25-point limit can be changed with `--forest-top-n`.
- **Pair network:** Starts with the same `D` (or the fallback set), ranks candidate pairs by decreasing `|TMLE log interaction|`, and processes them greedily. It adds a pair only while fewer than 25 edges have been kept **and** both genes currently have degree below 4; then it increments both degrees. A strong pair can therefore be omitted because an endpoint has already reached degree 4, and fewer than 25 edges can remain if there are too few candidates or the degree cap prevents further additions. This is a sparse readability rule, not a global optimization or a second statistical test. Red/blue marks sign, edge width shows relative absolute effect, and node size shows retained degree. It is intentionally not a graph of every significant pair.
- **Hypergraph (orders above 2):** Draws at most the eight strongest screened sets as outline polygons. Red/blue marks sign and line width shows relative magnitude. Polygon overlap has no extra statistical meaning.

6. Why the current workflow is faster

The speed improvement changes the implementation, not the question being estimated. The retired code processed one set at a time, queried a generic circuit integrator separately for each binary pattern, and repeatedly scanned held-out cells in Python for counts, diagnostics, and TMLE.

The current code batches gene sets and patterns, uses exact tree message passing, builds contingency tables with tensors, and solves the TMLE updates in batches. It still calculates exact requested marginals for the fitted PC and uses the same universal least-favourable TMLE update. It is not expected to match the retired Cirkit checkpoint numerically, because the PC class, tree learning, and fitted checkpoint are different.

The code can enumerate all panel sets for the listed orders, but the `2^r` pattern table gets sparse as `r` grows. That loss of support is the practical limit for unregularized high-order inference.

B Complete 999-gene analysis guide (verbatim)

```
# 999-gene marginal-estimand analysis

This guide explains the scalable analysis that considers all 999 genes. It can calculate every pairwise interaction,
↪  but it cannot practically test every possible three-gene or larger set. Higher-order results therefore come from a
↪  clearly labelled screening workflow and should be treated as exploratory.

## At a glance

The workflow has two different scopes:

- **Pairs:** calculate all 498,501 unordered gene pairs exactly under the fitted PC, then perform cross-fitted TMLE
↪  inference where the data support it.
- **Orders 3 and above:** use the pairwise results to select a manageable set of candidate groups, then evaluate
↪  those candidates exactly. This is not an exhaustive all-gene search and it is not confirmatory inference.

An interaction measures association among binary gene-expression states. It does not by itself show direct
↪  regulation, a biological mechanism, or causation.

## Data and saved files

The source CSV contains 11,950 cells and 1,000 numeric columns. The unnamed first column is constant metadata and is
↪  removed. All 999 remaining named columns are kept in their CSV order and must be binary (0/1). The current
↪  analysis uses every cell and every gene; it applies no cell or gene filter.

Development options such as `--max-samples` and `--max-genes` change the data fingerprint. A checkpoint or cache from
↪  a development run therefore cannot silently be used with the full dataset.

The fingerprint hashes the ordered gene names and their binary values. The code requires an exact match before
↪  loading a checkpoint or reusing a pair cache. This prevents a changed input file or reordered columns from
↪  producing an apparently valid but incorrect analysis.

| File | What it contains |
| --- | --- |
| `output/pc_architecture_sweep_999_genes.csv` | Results for all architecture-selection candidates. |
| `output/pc_architecture_selected_config_999_genes.json` | Settings selected by validation likelihood. |
| `output/cross_fitted_pcs_999_genes.pt` | Two trained fold-specific PCs and run provenance. |
| `output/cross_fitted_pcs_999_genes_marginal_estimand_pair_cache.pt` | Exact marginal PC pair probabilities and
↪  held-out pair counts. |
| `output/cross_fitted_pcs_999_genes_marginal_estimand_pairwise_tmle.pt` | Marginal-TMLE results for all `C(999,2) =
↪  498,501` pairs. |
| `plots/*marginal_estimand_pairwise_tmle_{overview,forest,network}.png` | Marginal pairwise summary figures. |
| `*_marginal_estimand_order{r}_candidates.pt`, `*_marginal_estimand_order{r}_tmle.pt` | Screened marginal
↪  higher-order candidates and their results. |

Run the usual workflow from the repository root:

~~~bash
python sweep_pc_architectures.py --analysis 999
python train_cross_fitted_pcs.py --analysis 999
python cross_fitted_interactions.py --analysis 999 --interaction-order 2 --rebuild-cache
python cross_fitted_interactions.py --analysis 999 --interaction-order 3
~~~

## 1. Select the PC model

Architecture selection is separate from interaction inference. With `seed=0`, the 11,950 cells are randomly divided
↪  into 9,560 selection-training cells and 2,390 validation cells. Each candidate trains for 20 epochs on the first
↪  group and is ranked by mean validation negative log likelihood (NLL) on the second. Lower NLL is better.

The selection grid has 39 candidates:

- four architectures: factorized, latent-class, mixture-Chow--Liu, and latent-class-residual-tree;
- 4, 8, 16, or 32 components for non-factorized models (the factorized model has one);
- AdamW learning rates of 0.005, 0.01, and 0.02; and
- a batch size of 1,024 cells, weight decay of `1e-5`, and seed 0.

For each tree-based candidate, the sweep learns one smoothed Chow--Liu maximum-mutual-information tree from the
↪  selection-training cells only. It roots the tree at a high-entropy gene and reuses that topology for the sweep's
↪  tree candidates. The final cross-fitted PCs relearn a separate topology from their own training halves.
```

```
The saved winner was:

| Selected setting | Value |
| --- | --- |
| Architecture | latent-class-residual-tree |
| Components | 32 |
| AdamW learning rate | 0.01 |
| Weight decay | `1e-5` |
| Validation NLL | 241.895630 |
| Trainable parameters | 33,998 |

The selected PC has 32 mixture components. Each component has its own baseline Bernoulli logit for every gene.
↪  Tree-edge residual logits are shared across components:

~~~text
logit p(X_j=1 | X_parent(j)=u, Z=z)
    = base_logit[z,j] + residual_edge_logit[j,u]
~~~

In plainer terms, the components can represent broad cell-state patterns, while the shared tree residuals represent
↪  compact gene-to-gene dependence. The root uses its component-specific baseline, and softmax-normalized weights
↪  combine the components. The parameter count is `32 x (999 + 1) + 2 x 999 = 33,998`.

No interaction result, p-value, or q-value was used to choose this model.

## 2. Cross-fit the final models

The selection sweep does not describe the final checkpoint training in full. The saved checkpoint was trained as
↪  follows:

| Checkpoint setting | Recorded value |
| --- | --- |
| PC / components | latent-class residual tree / 32 |
| Cross-fitting folds | 2 |
| Training / held-out cells per fold | 5,975 / 5,975 |
| Epochs | 10 |
| Training minibatch | 1,024 cells |
| AdamW learning rate / weight decay | 0.01 / `1e-5` |
| Fold seeds | 0 and 1 |
| Device | CPU |
| Training time | 11.31 s model-fit/save timer; 12.16 s complete fresh training command |
| Final training NLL, folds 1 and 2 | 246.000771, 246.297290 |

Each epoch shuffles cells and AdamW minimizes the mean NLL. A minibatch always contains **cells**; it does not split
↪  the 999 genes into smaller analyses.

Cross-fitting means each cell is assessed by a model that did not train on it:

~~~text
All 11,950 cells -- seeded split --> F1 (5,975) + F2 (5,975)

PC_1 learns from F2  --> predicts probabilities and targets F1
PC_2 learns from F1  --> predicts probabilities and targets F2

Combine fold estimates by held-out sample size and pool EIF moments for uncertainty.
~~~

Together, the two held-out folds include every cell exactly once.

## 3. Calculate all pairs with one exact cache

There are 498,501 unordered pairs, but the workflow does not fit or separately query 498,501 PCs. With
↪  `--rebuild-cache`, it checks the data/checkpoint fingerprint, loads the two fold PCs, and creates a cache
↪  containing the following for each fold:

1. `p_one[i] = p(X_i=1)`, a length-999 vector;
2. `p11[i,j] = p(X_i=1, X_j=1)` for the upper triangle;
3. held-out `count_one[i]` and `count11[i,j]`; and
4. the held-out fold size.

For this tree-mixture PC, one-gene and two-gene probabilities have exact analytic solutions. The code propagates
↪  component means and covariances along the tree, averages them using the mixture weights, and builds one 999 x 999
↪  pair-probability matrix per fold. It never enumerates the impossible `2^999` full joint state space.
```

Observed counts are also calculated in bulk. One-gene counts are column sums; all 11 counts come from `X.T @ X`, an optimized dense matrix multiplication. The remaining three cells for a pair follow from identities:

~~~text
p00 = 1 - p_i - p_j + p11     n00 = n - n_i - n_j + n11
p01 = p_j - p11               n01 = n_j - n11
p10 = p_i - p11               n10 = n_i - n11
p11 = p11                     n11 = n11
~~~

Recorded fresh end-to-end runtime

The current saved run trained a new two-fold checkpoint from the full source CSV, then forced a new pair-cache build before all-pairs TMLE. No checkpoint, cache, or pairwise result from an earlier run was read. It used `minimum-cell-count=0`, `minimum-probability=1e-12`, 64 bisection iterations, and default plots.

Stage	Time	What the timed stage contains
Fresh training command	12.16 s	Complete external Python-command time, including startup/imports, input loading, both PC fits, and checkpoint saving. The checkpoint's internal model-fit/save timer was 11.31 s.
Build and save the pair cache	1.24 s	Program stage timer within the fresh pairwise command: restore the newly trained PCs; generate exact one- and two-gene probability tables and held-out count tables; and write the cache `.pt` file.
All-pairs TMLE, final save, and plots	1.30 s	Program stage timer within the fresh pairwise command: construct 498,501 four-cell tables; do two-fold TMLE, SE/p-value/BH calculation; write the final pairwise `.pt` result; print the report; and render the overview, forest, and network images.
Fresh pairwise command, complete	3.50 s	Complete external Python-command time for cache construction, TMLE, final save, and plots, including startup/imports, input loading, validation, result reload, and wrapper overhead. Its internal interaction timer was 2.65 s.
Fresh sequential pipeline	15.66 s	`12.16 + 3.50`: one new PC fit followed by one new cache build and the all-pairs analysis.

The complete external command times are the `real` values from `/usr/bin/time -p`; the two substage rows are the program's own logged timers. Thus the cache-construction row is the recorded **data-generation and cache-saving** time, while the all-pairs row is the recorded **TMLE, final-result saving, and plotting** time. The log does not separately time each operation inside those rows, so it would be misleading to assign finer-grained values. A later marginal-estimand run can reuse a matching marginal-estimand cache, but this benchmark run did not.

Why this is faster than the legacy workflow

The retired 12-gene code repeatedly used a generic circuit-integration query for each target, binary pattern, and fold. It also scanned cell rows in Python for counts and TMLE diagnostics.

The scalable workflow instead uses a calculation specific to pairwise marginals in this model, matrix arithmetic for observed counts, and vectorized operations across 498,501 four-cell tables. This is an algorithmic speed-up, not evidence that hundreds of thousands of PCs were fitted. Results are not expected to reproduce the retired implementation exactly, because the PC class, tree learning, and saved checkpoint differ.

4. Pairwise interaction, TMLE, and support

For a pair, the target is the log odds ratio:

~~~text
theta(p) = log[p00 p11 / (p01 p10)]
~~~

Zero means no interaction on this binary odds-ratio scale. Positive and negative values mean positive and negative association. This is not a causal estimate and does not establish a direct biological mechanism.

What TMLE does

For each cross-fitted fold, TMLE starts with the PC's four-cell table `p_0` and makes a targeted update:

~~~text
p_epsilon(a) = p_0(a) + epsilon s(a),    s = (+1, -1, -1, +1)
~~~

`epsilon` must keep all four probabilities strictly positive. Its allowable interval is:

~~~text
max(-p00,-p11) < epsilon < min(p01,p10)
~~~

```
After shifting the endpoints slightly inward, the code runs 64 vectorized bisection steps to solve:

~~~text
sum_a p_n(a) s(a) / p_epsilon(a) = 0
~~~

`p_n` is the observed distribution in that held-out fold. This is the efficient-influence-function (EIF) score
↪  equation: TMLE chooses the update whose empirical EIF mean is zero. The EIF is **not** minimized.

At the updated table `p_star`, the centered EIF is:

~~~text
H(a) = s(a)/p_star(a) - E_pstar[s(A)/p_star(A)]
~~~

Fold log estimates are weighted by held-out sample size. Pooled EIF values give the SE, a 95% Wald confidence
↪  interval, and a normal-approximation p-value. Benjamini--Hochberg (BH) then adjusts p-values over the supported
↪  pair family.

### When the table reports uncertainty

The result table distinguishes two statuses:

- **Targeted:** The initial table is finite and strictly positive, and bisection has a valid root in both folds. It
↪  has a finite point estimate.
- **Supported:** The result is targeted and, in each fold, the smallest of the four observed counts is at least
↪  `--minimum-cell-count`, every initial model probability is at least `--minimum-probability`, and the table is
↪  finite.

Only supported pairs receive an SE, CI, p-value, and q-value. A finite targeted pair that fails support is
↪  descriptive only. Support is an unregularized positivity and stability requirement; it is not a score of
↪  biological importance.

> **What "supported unregularized" means here.** Inference uses the raw held-out four-cell table. It does not add a
↪  pseudocount, continuity correction, probability floor, ridge penalty, or variance regularization to repair an
↪  empty or rare cell. The thresholds are gates: they decide whether inference is shown, but they do not modify a
↪  count or clip a probability.
>
> In the current rerun, `minimum-cell-count=0`. The held-out count rule therefore does not exclude a pair with an
↪  empty 00, 01, 10, or 11 cell; it still applies independently to F1 and F2 rather than pooling counts across all
↪  11,950 cells. In every fold, the PC must also assign all four cells a probability of at least `1e-12`; lower
↪  values are excluded, not raised to that threshold.
>
> "Targeted" is weaker than "supported." A finite, strictly positive model table can have a valid TMLE solution even
↪  if one held-out cell is zero or otherwise below the threshold. Such a pair deliberately receives no SE, CI,
↪  p-value, or q-value.

The most recent table used `minimum-cell-count=0` and `minimum-probability=1e-12`. Of 498,501 pairs, 389,779 have
↪  finite targeted estimates and all 389,779 are supported. The remaining 108,722 pairs are not targeted because
↪  TMLE did not yield a finite point estimate in both folds; none is excluded by the held-out count or `1e-12`
↪  model-probability threshold.

Lowering the minimum count increases coverage, but it permits inference based on smaller cells. That is a scientific
↪  design choice, not a speed optimization.

### What the BH q-value in the overview plot means

BH is applied **only** to the 389,779 supported pairs in this run; targeted-but-unsupported pairs have no p-value or
↪  q-value and are not counted.  Let `m = 389,779`, let `p_h` be the two-sided Wald p-value for pair `h` under its
↪  null hypothesis of zero log interaction, and write the ordered p-values as

~~~text
p_(1) <= p_(2) <= ... <= p_(m).
~~~

For a chosen false-discovery-rate (FDR) level `alpha`, the BH procedure finds the largest rank `k` for which

~~~text
p_(k) <= k * alpha / m,
~~~
```

then calls ranks `1, ..., k` discoveries. The reported value for the pair at rank `i` is the corresponding monotone BH-adjusted p-value:

~~~text
q_(i) = min_{j=i,...,m} min(1, m * p_(j) / j).
~~~

The value is then returned to its original pair. Thus `q_h <= alpha` means exactly that pair `h` belongs to the BH discovery set if BH is run at level `alpha`; equivalently, `q_h` is the smallest BH FDR level at which it would be selected. The name "q-value" here means this BH-adjusted p-value, not a posterior probability or a local false-discovery rate.

In the overview plot, pair `h` has vertical coordinate

~~~text
y_h = -log10(q_h),
~~~

with values above 50 visually capped at 50. Higher points have smaller BH-adjusted p-values: `q=0.05`, `0.01`, and `0.001` map to about `1.301`, `2`, and `3`, respectively. The horizontal line is therefore at `-log10(0.05) = 1.301`; points on or above it meet the BH `q <= 0.05` criterion. The x-coordinate and colour describe the estimated TMLE log interaction and its sign, respectively. They do not change the q-value, which comes from a two-sided p-value.

BH controls the FDR of the *set* selected at a fixed cutoff, where

~~~text
FDP = V / max(R, 1),       FDR = E[FDP],
~~~

`R` is the number of BH discoveries and `V` is the (unobserved) number of those discoveries whose null hypotheses are true. Under valid null p-values and the usual BH dependence conditions (independence, or positive regression dependence on the true nulls), selecting every pair with `q <= 0.05` gives `FDR <= 0.05`. It does **not** say that five percent of a particular subset, one plotted point, or one named edge is false, and it does not establish causation. The pair tests share cells and genes, so they are dependent; the standard BH guarantee is not a guarantee under arbitrary dependence. A confirmatory analysis needing protection under arbitrary dependence would need a different prespecified correction, such as Benjamini--Yekutieli.

References: Benjamini and Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B* 57(1), 289--300 (1995), [doi:10.1111/j.2517-6161.1995.tb02031.x](https://doi.org/10.1111/j.2517-6161.1995.tb02031.x); Benjamini and Yekutieli, "The Control of the False Discovery Rate in Multiple Testing under Dependency," *Annals of Statistics* 29(4), 1165--1188 (2001), [doi:10.1214/aos/1013699998](https://doi.org/10.1214/aos/1013699998).

The default display screen additionally requires `|TMLE log interaction| >= 0.25`; 132,177 pairs pass both cuts in the current run.

5. Read the pairwise table and figures

The pairwise table keeps every result: PC plug-in and TMLE log effects, SE, CIs, p-values, q-values, support flags, and the smallest count/probability. Exponentiate a pair's log effect to get its odds ratio. Consider its CI, q-value, raw support, and external validation before drawing a biological conclusion.

The three figures are intentionally readable summaries rather than complete graphs. They begin with pairs that are supported and have finite TMLE estimates and confidence intervals. With the default settings, their display set is

~~~text
D = {h : supported, finite estimate/CI, q_h <= 0.05,
         and |TMLE log interaction_h| >= 0.25}.
~~~

The two cuts have different jobs: `q <= 0.05` is the multiple-testing evidence screen, whereas `|log interaction| >= 0.25` is an effect-size screen (odds ratio at least `exp(0.25) ≈ 1.28`, or at most `exp(-0.25) ≈ 0.78`). If no pair satisfies **both** cuts, the forest and network fall back to all supported finite pairs, so a figure is still available. A fallback figure is therefore a ranking of supported pairs, not a set that passed both default cuts. The overview does not use this screen: it always includes every supported pair.

- **Overview:** Includes all supported pairs. The x-axis is the TMLE log interaction; the y-axis is `-log10(BH q-value)`, capped at 50 only for display. Red means positive and blue means negative. Dashed vertical lines mark +/-0.25; the horizontal line at 1.301 marks `q=0.05`. The BH family is all supported pairs, not only pairs passing the effect-size display cut.

```
- **Forest:** Separates `D` into negative estimates (`theta < 0`) and non-negative estimates (`theta >= 0`). Within
↪  each sign it orders by the estimate itself--most negative first on the left and most positive first on the
↪  right--and takes `min(25, number available)`. The two panels are selected independently, not balanced to contain
↪  25 each. Thus fewer than 25 points on a side means fewer than 25 display-set pairs have that sign (or, after the
↪  no-hit fallback, fewer than 25 supported finite pairs have that sign). It does not mean that a 25th pair failed
↪  an extra confidence-interval or significance test. A sign with no eligible pair is labelled "None selected."
↪  Points are TMLE log effects, bars are 95% CIs, and the black line marks zero. The default 25-point limit can be
↪  changed with `--forest-top-n`.
- **Network:** Starts with the same `D` (or the fallback set), ranks candidate pairs by decreasing `|TMLE log
↪  interaction|`, and processes them greedily. It adds a pair only while fewer than 25 edges have been kept **and**
↪  both of its genes currently have degree below 4; it then increments both degrees. A pair can therefore be absent
↪  despite passing the two display cuts because a gene in the pair already has four stronger retained neighbours.
↪  Fewer than 25 edges can remain if candidates run out or degree caps block the rest. These are sparse readability
↪  rules, not a global graph optimization or extra statistical tests. Red/blue marks sign, edge width shows relative
↪  absolute effect, and node size shows retained degree.

The all-gene network may contain two or more **disjoint components**.  Only genes incident to a retained edge are
↪  drawn, and the code does not require retained edges to form a connected graph.  Separate components arise when
↪  high-ranking retained pairs do not share genes, when potential bridging pairs fail the q-value or effect-size
↪  cuts, or when a bridge is skipped after one of its endpoints reaches the degree cap (or the 25-edge limit is
↪  reached).  The plotting code deliberately lays disconnected components apart so their labels and edges remain
↪  readable.  Their separation is not evidence that the gene groups are biologically independent, nor evidence that
↪  no association exists between them; it is a consequence of screening and sparsification.

## 6. Screened higher-order interactions

For `--interaction-order r >= 3`, the code first runs or reuses the all-pair stage. It then chooses a manageable
↪  candidate set:

1. keep pairs that are supported and finite, have `q <= 0.05`, and have `|log interaction| >= 0.25`;
2. for each gene, retain its 20 neighbours with the largest absolute pair effect;
3. by default, keep an edge only if both genes selected each other (`edge-policy=mutual`); `edge-policy=union` keeps
↪  an edge if either endpoint selected it;
4. enumerate `r`-cliques in the retained graph, stopping with an error above `max-candidates=100,000`; and
5. save each candidate's smallest absolute constituent-pair effect and largest constituent-pair q-value.

For every selected candidate, the PC then calculates all `2^r` binary-pattern probabilities exactly by batched tree
↪  message passing. The held-out patterns are counted with binary coding and `bincount`. The same TMLE update,
↪  support checks, pooled EIF variance, p-value, and BH calculation are then applied.

The higher-order target is:

~~~text
theta_r(p) = sum_{a in {0,1}^r} (-1)^(r-sum(a)) log p(X_S=a)
~~~

This is not a complete all-gene high-order search. The same pairwise data select candidates and are later used to
↪  estimate them. The saved higher-order result therefore carries an explicit exploratory warning. Do not present
↪  its q-values as confirmatory FDR evidence; use an independent confirmation split or a selection-aware design.

## 7. Why exhaustive high-order scans do not scale

The number of possible gene sets grows very quickly:

| Order r | All-gene set count `C(999,r)` |
| ---: | ---: |
| 2 | 498,501 |
| 3 | 165,668,499 |
| 4 | 41,251,456,251 |
| 5 | 8,209,039,793,949 |

Each order-`r` set also has `2^r` binary pattern cells. A general tree query takes work roughly proportional to
↪  `batch size x 2^r x 999 x 32` per fold, plus count construction. TMLE then performs 64 bisection updates across
↪  those `2^r` cells. Larger batches reduce loop overhead, but they use more memory and do not reduce the number of
↪  sets.

- The exact analytic pair cache makes a complete order-2 analysis practical on CPU. Its special pairwise identities
↪  do not exist for arbitrary larger sets.
- An exhaustive order-3 scan is not practical in this implementation on a CPU or a typical GPU. It starts with 166
↪  million sets before accounting for eight patterns, two folds, PC queries, counts, TMLE, or result storage. A
↪  multi-field results table alone would need many tens of gigabytes.
- Orders 4 and 5 are much larger still. A normal GPU cannot remove the combinatorial `C(999,r)` growth or the `2^r`
↪  pattern growth.
```

```
- A CUDA GPU can speed model fitting and a **screened, bounded** set of higher-order candidates. With enough memory,
↪  it can also allow a larger candidate batch. It cannot make exhaustive high-order inference feasible.
- On Apple MPS, pairwise and higher-order TMLE run on CPU because the implementation needs reliable float64
↪  arithmetic and MPS float64 support is limited.

The practical approach is to calculate all pairs exactly, screen using predeclared q-value, effect-size, and degree
↪  limits, cap the candidate set, inspect support carefully, and independently confirm selected higher-order
↪  findings.
```